Hands That Speak

Mohsin Murtaza	Muhammad Zain Ul Hassan	Aiman Manzoor
051-21-135133	051-21-135346	051-21-135159



Supervised by

Ms. Sana Yousuf

A Final Year Project Proposal Report is
Submitted in Partial Fulfilment of the
Requirements for the Degree of
Bachelor of Science in **Computer Science**

Department of Computing & Technology

Iqra University

2025

Certificate

We hereby accept the work contained in this report titled: Hands That Speak, as a confirmation to the required standards for the partial fulfilment of the degree of Bachelors of Science in Computer Science.

Internal Examiner

External Examiner

Project Supervisor

Head of Department

Declaration

We, Muhammad Zain-Ul-Hassan , Aiman Manzoor , Mohsin Murtaza , solemnly declare that research work presented in my undergraduate project, as the case, titled "Hands That Speak" is solely our research work with no significant contribution from any other person or Generative Al Tools. Small contribution helps wherever taken has been duly acknowledged/cited and that complete project/thesis has been written by us in accordance with the latest plagiarism policy declared by HEC and our respective university in-line with the policy for use of Generative Al Tools.

We understand the zero- tolerance policy of the HEC and Iqra University Islamabad Campus towards plagiarism. Therefore, we as an Author of the above titled project/thesis declare that no portion of my project/thesis has been plagiarized, and any material used as reference is properly referred cited. We undertake that if we are found guilty of any plagiarism in the above titled project even after award of undergraduate degree, the University reserves the right to withdraw/revoke our degree and that HEC and the University has the right to publish our names on the HEC/University Website on which names of students are placed who submitted plagiarized project.

Muhammad Zain Ul Hassan
151-19-135346
Aiman Manzoor
151-19-135159
Mohsin Murtaza
151-19-135133

Dedication

We wholeheartedly dedicate this project to our families and loved ones. Your unwavering support, constant encouragement, and deep belief in our potential have been the driving forces behind every step of this journey. Throughout the challenges and triumphs, your presence has been a steady source of strength, motivation, and inspiration. To our parents, guardians, and mentors your wisdom, sacrifices, and enduring faith in us have shaped our character and fuelled our determination. You have taught us the value of perseverance, integrity, and hard work, lessons that have guided us not only in this project but in every aspect of our lives. To our friends and loved ones thank you for your patience, understanding, and emotional support, even during times when we were unavailable or overwhelmed. Your encouragement helped us stay focused and resilient, reminding us of the bigger picture. This project stands not only as a reflection of our individual efforts and dedication but also as a testament to the love, values, and guidance you have instilled in us. Without your support, this achievement would not have been possible. We are forever grateful.

Acknowledgements

We wish to thank our parents for their support, encouragement, and trust in us. Their moral and financial support has formed the backbone of our journey. We are grateful to our Final Year Project Coordinator, who is committed to guiding us through our final year project. We would also like to extend our thanks to our supervisor, Ms. Sana Yousuf, who dedicated time and effort into guiding us throughout. Her insights and encouragement have been invaluable to bring this project to life.

Abstract

This project introduces Hands That Speak, an agile mobile app engineered for Continuous Sign Language Translation (CSLT) specifically for Pakistani Sign Language (PSL). It closes the communication divide between Deaf users and the hearing world by turning sign language into fluent Urdu text, correctly respecting grammar and context, all powered by state-of-the-art deep learning. Our approach weaves together video prep, joint-key point detection, and cutting-edge transformer families Swin, PGViT, and SignFormer then links them to a multilingual mBERT engine for context-aware, semantically dense translation. Central to our contribution is a PSL corpus we compiled by augmenting existing publicly available datasets with volunteer-recorded clips. To combat limited data and heighten model robustness, we applied aggressive data augmentation and class-wise concatenation, injecting rich variation into gesture samples. The architecture is tuned for real-time performance, encoding both the spatial and temporal dynamics of PSL through stacked transformer blocks, ultimately producing precise text. We measure success with accuracy, F1 score, and latency, ensuring our system is both precise and practical.

Table of Contents

Certificate i

Declaration ii

Dedication iii

Acknowledgements iv

Abstract v

Chapter 1: Introduction 1

1.1. Introduction 2

1.2. Overall Description 2

1.3. Objectives 3

1.4. Problem Description 3

1.5. Methodology 4

1.5.1. Exploratory Research 4

1.5.2. Analytical Techniques 4

1.5.3. Advantages 4

1.5.4. . Features and Functionalities 4

1.5.5. Roles and permission of Users Our Responsibility 5

1.5.6. Admin access 5

1.5.7. Quality Assurance 5

1.5.8. Testing and Quality 5

1.5.9. Experiences 6

1.5.10. Geographic Expanding 6

1.6. Business Context 6

1.6.1. Partnerships 6

1.6.2. Marketing 6

1.7. Users Classes and Characteristics 6

1.7.1. User (Characteristics) 7

1.7.2. User Action 7

1.7.3. Guest 7

1.7.4. Admin 7

1.8. Operating Environment 8

1.9. Assumptions and Dependencies 8

1.10. Functional Requirements 8

1.10.1. Video Inputs 8

1.10.2. Gesture Detection and Classification 9
1.10.3. Pause Detection 9
1.10.4. Multi-Gesture Recognition 9
1.10.5. Model Training 9
1.10.6. System Performance 9
1.10.7. Error Handling 10
1.10.8. Privacy and Security 10
1.10.9. Online and Offline Functionality 10
1.11. Non-functional Requirements 10
1.11.1. Performance Requirements 10
1.11.2. Safety Requirements 11
1.11.3. Security Requirements 11
1.12. External Interface 11
1.12.1. Use Cases 11
1.12.2. Software Interface 12
1.12.3. Hardware Interface 12
1.12.4. Communication Interface 13
1.13. System Features 13
1.13.1. Real-Time Gestures 13
1.13.2. Machine Learning 14
1.13.3. Grammatical and Contextual 14
1.13.4. System Accessibility 14
1.13.5. Multimodal Gestures 14
1.14. Software Quality Attributes 14
1.15. Scenarios 15
1.16. Report Structures 15
Chapter 2: Literature Review 16
2.1. Introduction 17
2.2. Training Models 17
2.2.1 Terminologies 24
2.2.2 Categorization of Existing Techniques 25
2.2.3 Limitations 26
2.3. Proposed Improvements 27
2.4. Summary 28
Chapter 3: System Design 29
3.1 Introduction 30

3.1.1. Purpose 30
3.1.2 System Overview 30
3.1.3. Design Map 31
3.2. Design Consideration 32
3.2.1. Assumptions 32
3.2.2. Constraints 32
3.2.3 Design Structure 33
3.2.4 Risks and Boundaries 33
3.2.5 Volatile Risks 33
3.2.6. Overview 34
3.3. System Architecture 34
3.4. High Level Design 34
3.4.1. Use Case Diagram 34
3.4.2. Full Functions on Use Case 37
3.4.3. Context Diagram 39
3.4.4. Data Flow Diagram (DFD) 39
3.5. Low Level Design 40
3.5.1. Sequence Diagram 40
3.5.2. Flow Chart 42
3.6. User Interface Design 44
3.6.1. Authentication Screens 44
3.8. Summary 49
Chapter 4: Implementation 50
4.1 Discussion 51
4.1.1 Implementation Design 51
4.1.2 Challenges Encountered 51
4.1.3 Problem Solving 51
4.2 Research Methodologies 52
4.2.1 Agile Research 52
4.2.2 Iterative Method 52
4.2.3 Testing and Validation-Driven Research 52
4.3 Implementation of Tools 52
4.3.1 Frameworks and Libraries 52
4.3.2 Development Tools 53
4.3.3 Libraries Tools 53
4.4 Implementation Features 53

4.4.1 Algorithms and Methods 53
4.4.2 Datasets 56
4.4.3. Screenshots: [35], [36] (MobileNetV2, user study) 58
Chapter 5: Testing 61
5.1. Testing Techniques Employed for This Project 62
5.1.1. Pre-processing & Environment Testing 62
5.1.2. Model Testing and Comparisons 62
5.1.3. Training Strategy Evaluation 63
5.1.4. Evaluation Metrics 63
5.1.5. Limitations and Observations 63
5.3. Test Results 64
5.4. Summary 65
Chapter 6: Data Analysis and Results 66
6.1. Methodology 67
6.2. Empirical Study Structure 67
6.3. Metrics for Result Comparison 68
6.4. Performance Evaluation 68
6.4.1. SignFormer Results [24], [7], [16] 69
6.4.2. Swin Transformer Results [24], [7], [16] 69
6.4.3 Comparison 70
6.4.4 Confusion matrix Sign Former 70
6.4.5 Confusion matrix Swin Transformer 71
6.5. Summary 71
Chapter 7: Conclusion and Future Work 72
7.1. Contributors 73
7.1.1. Contributions 74
7.2. Findings 75
7.3. Future work 75
7.3.1 Improvement within the Current System 75
7.3.2 Next-Generation System Design 76
References 77

List of Tables

Table 2. 1: Convolutional Neural Networks ……. 17
Table 2. 2: Transformers ……..…….. 19
Table 2. 3: DINO …….…… 20
Table 2. 4: DINO v2 ……..…… 21
Table 2. 5: Swin Transformer ……..…….. 22
Table 2. 6: SignFormer ……..…..... 23
Table 2. 7: PGVT …….…… 24
Table 2. 8: Techniques in CSLT ……..…….. 27
Table 3. 1: Use Case ……..……..... 36
Table 3. 2: Full Functions on Use Case ……..…...… 37
Table 4. 1: Augmentation Techniques ……..……. 57
Table 5. 1: Test cases and their results. …… 64
Table 5. 2: Test results ……..……. 64
Table 6. 1: SignFormer results. ……..…… 69
Table 6. 2: Swin Results ……..…….. 69
Table 6. 3: Comparison between diff models. ……...……..…….. 70

List of Figures

Fig. 3.1 System Architecture 33
Fig. 3.2 Use Case Diagram 35
Fig. 3.3 Entity Relation Diagram 38
Fig. 3.4. Context Diagram 39
Fig. 3.5. Data Flow Diagram 39
Fig 3.6 Real time translation 40
Fig 3.7 Educational support 40
Fig 3.8 Health care support 41
Fig. 3.9 Customer care support 41
Fig. 3.10. Flow Chart 42
Fig 4.1. Design methodology 56
Fig. 4.2. Labelled Frame 59
Fig. 4.3 Plotting Important Features 60
Fig. 4.4 Hand Landmark 60
Fig. 4.5 Extracted Labels 60

Chapter 1: Introduction

1.1. Introduction

Our application Continuous Sign Language Translation [CSLT] have the ability to change people lives. We took this initiative under our own arms to help the people that are unable to communicate fluently with other people due to their speaking disability. With the help of this app we really tried to help the community of people that cannot communicate with the rest of the world without any complications. The basic goal of ours was to narrow down the barriers between people who cannot express themselves freely to other people.

1.2. Overall Description

The team developed an application intended to facilitate communication with individuals who had speaking disabilities. The goal was to enable these individuals to interact more freely, allowing others to understand them without barriers, thus serving a greater social good. The group members collaborated because they shared a common objective and were motivated by the desire to support an often-overlooked minority within the community. Recognizing that very few initiatives addressed the needs of people who could not speak, the project aimed to contribute meaningfully to their future.

In the development process, the team utilized a Convolutional Neural Network (CNN) in conjunction with Long Short-Term Memory (LSTM) networks, which initially appeared to be the most effective approach. However, the team remained open to adopting alternative models as the project progressed, ensuring adaptability. A key feature of the application included mechanisms for collecting frequent user feedback, enabling iterative improvements and continuous updates based on real-world usage.

While previous efforts in sign language recognition existed, few had focused on continuous sign language translation. As implied by the term “continuous,” this approach processed sequences of frames in near real-time, aiming to reduce communication barriers more effectively and create a seamless interaction experience.

1.3. Objectives

- Enable non-verbal children, especially in early education, to communicate their needs through gesture recognition technology.
- Develop a video-based system (CSLT) to translate hand gestures and facial expressions into meaningful text or speech.
- Continuously improve system performance based on user feedback to minimize communication barriers over time.
- Provide a free and accessible solution that only requires a smartphone or smart device for usage.
- Design an intuitive and user-friendly interface to increase adoption among people unfamiliar with assistive technologies.

1.4. Problem Description

The communication between people who can actually speak and people who have speaking disabilities is challenging. Sign language is one of the solutions but learning the hand gestures, body movements is time consuming and complex for people to understand.

CSLT is a solution for this matter by translating sign language from video in real time. This app processes each frame of a video to detect body movements, hand gestures, hand movements and then translates it into meaningful contextual sentences which narrows the communication barrier.

Similar to any other spoken language there are meaningful pauses between words or sentences which helps to understand the contextual meaning. In sign language these pauses also play an important role and also allows the app to understand when one movement has ended and the other one started.

The program will also have the ability to understand that there are different meanings of one word depending on the basis of context and the program can identify on the base of gestures that correspond with the correct meaning of the sentence. This will allow the app not just to

understand the gestures but also the structures and flow of sentences. Continuous Sign Language Translation [CSLT] will also improve to detect the pauses between gestures and exactly sync the gestures to words or sentences which would make sure that the words have actual meaning with the context of the sentence.

By tackling all the problems our app will be more effective and accurate than any other application on the basis of sign language.

1.5. Methodology

In this project we will follow the mixed approach including combining exploratory research with analytical techniques.

1.5.1. Exploratory Research

- Focuses on studying the differences in the dataset of Pakistani Sign Language.
- Focuses on detecting pauses between word gestures to translate with better accuracy.

1.5.2. Analytical Techniques

Utilizes pretrained models and transformers for gesture detection and feature extraction to provide translations.

1.5.3. Advantages

- User friendly
- Changes can be performed
- Bridge between people with speaking or hearing disability and people who can speak.

1.5.4. . Features and Functionalities

- Combining exploratory research on Pakistani Sign Language and different techniques using pretrained models.
- Dataset of Pakistani Sign Language will be utilized to train the model.

- Utilizes pre-processing techniques on the data to extract frames and enhance them for processing.
- Integrates near to real time contextual analysis, enabling sentences with proper meaning.
- To resolve ambiguity and accurately translate multiple gestures for the same word Contextual analysis will be used.

1.5.5. Roles and permission of Users Our Responsibility

We took the responsibility to ensure that not everybody could access the main developer side and can make changes or manipulate the information.

1.5.6. Admin access

The AI models of our app can be accessed by all our group members to constantly update the model on new dataset and videos being captured by the users.

1.5.7. Quality Assurance

The design of our app will be simple and easy so that users can easily understand utilize our app to full extent.

1.5.8. Testing and Quality

There are several methods for assurance of testing and quality to make it more efficient:

- Test Planning
- System Testing
- Performance Testing
- Security Testing
- Maintenance Testing

1.5.9. Experiences

To enhance our app we have the support of a professional teacher that will support us.

1.5.10. Geographic Expanding

First our priority is in Islamabad and Rawalpindi. But as we progress it will expand the geographical areas to help people on a larger scale in Pakistan.

1.6. Business Context

This app usage can be applied to any field that makes use of gesture recognition, but mainly in the healthcare and education sector. It can help individuals with hearing disabilities in order to communicate with the rest of the community in near to real time. Social life of such individuals can be enhanced with better communication and workplace inclusion.

1.6.1. Partnerships

The use of this app is much needed in the classes of kindergarten at schools. That is the reason we will collaborate with schools who are willing to invest in us and trust our app.

1.6.2. Marketing

We will use online marketing of our app by putting it on promotion pages and also through some schools that we will collaborate with so that they can advertise our app and more people will get aware of it with the assistance of people.

1.7. Users Classes and Characteristics

Our app will have different classes of users and have different access of information depending upon there selected class.

1.7.1. User (Characteristics)

- The User will have the access to the basic of the app what to translate and how to use the app.
- Users can give feedbacks what more can we do to improve the situation.

1.7.2. User Action

- Setup a profile and make an account.
- Observe and choose the options.
- Can donate to the app.
- Can access the history of donations and usage of app.
- We appreciate for them to use our app and keep updating them on our progress.

1.7.3. Guest

- Guest Users will have the access of app for a limited time.
- Guest Users cannot give feedbacks due to their lack of commitment to the app for not registering an account.

Guest Actions

- Not compulsory to create a profile.
- Access for a limited amount of time.
- Access to limited data.
- Will not be able to access the history of the app.
- Notify them about our registration process so they can participate and give feedback to us.

1.7.4. Admin

- The admins will have account access by entering their names and passwords.
- They can analyse all the given feedback and can take action on them and to get which of them is worth taking a look at and which of them are to be ignored. 3.The admin cannot mess with user account.

1.8. Operating Environment

In order for proper training of the models and model execution, a computer system with GPU is recommended. It should be able to work with video processing libraries, machine learning frameworks, ensuring seamless performance on Windows.

1.9. Assumptions and Dependencies

The following are the lists of assumptions and dependencies.

Assumption

- Users have access to modern day devices with high performing processors.
- Internet connectivity is provided for online authentication and cloud based services.
- App will be run on a compatible environment i.e. Android.
- Users' data will be safeguarded.

Dependency

- Flutter for app development with multiple packages like GetX, hl_image_picker etc for further functionality.
- Tools like Visual Studio Code, Android Studio, Google Colab for development and testing.
- Compliance with third party features like Firebase etc.

1.10. Functional Requirements

The following will be the functional requirements for our project.

1.10.1. Video Inputs

The app will be able to capture videos in high quality in a real time from a device. Each frame must analyse hand gestures and movements. The normal video input should be processed at a minimum of 30 frames per second (FPS) to ensure smooth gesture recognition.

1.10.2. Gesture Detection and Classification

To distinct each hand gestures from each other in a video frame. The use of pre trained models must be compulsory. Gestures classification should be accurate even under varying environmental conditions such as lighting and background noise.

1.10.3. Pause Detection

The app will detect pauses between gestures translating the body language and should confirm when a user pause is identified to signal the end of gesture.

1.10.4. Multi-Gesture Recognition

The App will be able to identify the multi gestures at one time and put the words in context to make a sentence. If in context a person throws multiple hand signs it will be able to detect all signs and translate only one which is in contextual order.

1.10.5. Model Training

The app will train machine learning models. It should support the use of pre-trained models, such as CNNs, and the ability to train new models on specific sign languages like PSL. The app will also include tools for data augmentation, model evaluation, and accuracy testing for the app. The training process should be flexible to accommodate new signs or in PSL as they emerge.

1.10.6. System Performance

The project includes a large number of datasets and multiple users are using it at one time so the back-end should support cloud-based deployment allowing system to handle high traffic and large amount of data. It should balance speed and performance.

1.10.7. Error Handling

The project should offer immediate feedback if the gesture is not recognized or show some error on the screen so that the user is notified that there is some issue and can reprocess it reperform the act. Immediate Feedback is essential for any app to work properly.

1.10.8. Privacy and Security

The system must encrypt and securely store video data to protect user privacy, adhering to data protection. Users will have the option to not share the use of data storage or sharing. These measures are essential for ensuring ethical use and building trust.

1.10.9. Online and Offline Functionality

Our app will need to work on internet connectivity because of its complexity. As internet connection is cut off, apps will stop working as most of the features depend on the internet.

1.11. Non-functional Requirements

Non-functional requirements also known as (NFRs) are methods to describe system characteristics. For example sustainability, safety, dependability, performance, and user friendliness. They perform as the boundaries on the architecture of the system across all the separate pending tasks. Here are our application and website non-functional requirements.

1.11.1. Performance Requirements

- **Real Time Processing:** The app is supposed to process frames in near to real time.
- **Computational Power:** High performing systems are required for better model training time for proper video processing without model crashing.
- **Accuracy:** App will be focused on better model accuracy, ensuring accurate translations.

1.11.2. Safety Requirements

The safety requirements for our project are thoroughly aiming to protect the user's data security and privacy, implementing measures to protect sensitive user information and details. Ensure user data, including video footage, is securely stored and transmitted, with encryption to protect personal information. To make strong security protocols to prevent unauthorized access to the system, particularly during near to real-time data processing. The system must be sustainable especially on smartphones for not to cause any over-heating.

1.11.3. Security Requirements

- **User Authentication:** User authentication will enable only registered users to access the translators.
- **Access Control:** Only specific people with access are allowed to access and change model setting and work with model accuracy.

1.12. External Interface

The given are the methods of External Interfaces.

1.12.1. Use Cases

There are three use cases of Interfaces.

1. Video-Interface

Used to manage real-time video input from the user, ensuring that gestures are captured within real time speed and that the frames are ready for translation.

2. Gesture-Interface

Performs when processing each video frame to identify specific hand movements capturing pauses and interpreting them to form complete sentences with contexts.

3. Translation-Interface

It is responsible for converting the signs into sentences with features like real-time feedback and text speech.

1.12.2. Software Interface

The list of software interfaces is under as follows:

Data Collection

- Allow users to upload and translate sign language videos.
- Provide tools for gesture tagging, frame annotations and extractions.
- Allows datasets for model training and evaluations.

Real-Time Gestures

- Takes input as live video and processes each frame for gesture recognition.
- Translates the given gestures into text or speech in real-time.
- Detects the pauses between hand gestures to form a sentence in context.

1.12.3. Hardware Interface

The given list is the hardware interface:

- **Processing and Display:** Requires high performance PC for model training and interfaces. Collaborate with smartphone for displaying real time recognition output.
- **Camera Interface:** To focus on specific areas of interest to capture hand gestures. Works with the camera of a device for a real time video capture.

1.12.4. Communication Interface

The app consists of diverse communication interfaces.

- Allows users to upload videos as input to capture in real time. Users can control pausing, stopping and starting of a video.
- It detects the hand movements and gestures in video frames. Detects pauses between gestures to form a sentence in context.
- Shows output in form of speech or text.
- Translates words in Urdu or English so user can understand easily.

1.13. System Features

The following are the given system features.

1.13.1. Real-Time Gestures

- Detects the movements of hand gestures and body language from video frames. Detects pauses between words to form a sentence accurately.
- Converts the recognized gestures in text language in real time.
- Spatial and temporal feature extraction is essential in translating continuous sign language. Transformers like:
- **Video Swin Transformer [18]**: Provides hierarchical spatiotemporal attention, enabling robust motion modelling.
- **SVFormer [19]**: A sparse vision transformer built for efficient spatiotemporal representation.
- **PGVT (Progressive Global-Local Vision Transformer) [20]**: Merges global context with local motion cues efficiently.
- **SignFormer [21]**: Tailored specifically for continuous sign language, employing deformable attention and dual-path fusion.

However, none of the above have been applied to PSL or Urdu-English sign language translation.

1.13.2. Machine Learning

- Uses Swin transformer models to extract key features of gestures.
- Analyse gestures context and assign them with body language to increase accuracy. Gather the datasets of PSL from Kaggle to train models on wide range gestures

1.13.3. Grammatical and Contextual

Predicts the words and put them in contextual order based on sign language. Considers surrounding Assumes hand signs for context to ensure accurate English translations.

1.13.4. System Accessibility

The app will be available on android smartphones. To make it public for general use. Easier for communication of nonsign language users, providing a user-friendly interface for seamless interaction.

1.13.5. Multimodal Gestures

- Observe not only just hand gestures but also body movements.
- It also scans the face features because it plays a vital role in communication.
- Tracks motion patterns to differentiate between similar gestures and ensure accurate translation of words and sentences.

1.14. Software Quality Attributes

Some of the software quality attributes are as follows.

- **Usability**

  - The app interface should be simple and user-friendly so it will be easier to use.
  - Every option needs to be properly organized into different categories so that the user has no trouble navigating through the app.

- **Availability**

    - The app must work properly and smoothly without any problems.
    - It will always be available for the person no matter wherever you are.

1.15. Scenarios

User Registration: Anyone who want to register to our app can be registered by giving required information without any complications.

Health Interaction: A deaf patient needs to communicate with a doctor on his disease symptoms but doctor is having difficulty understanding him. But this app will help in their situation.

User Interface: User friendly interface is inviting and allows the users to understand and perform the tasks easily.

Feedback: The users can give feedback if they are having difficulty or any other problems. Which will allow us to take effective initiatives to make improvements in that certain area.

Classroom: The children of kindergarten who are unable to speak are having difficulty while communicating with teachers but with our app their barriers will decrease to almost zero.

1.16. Report Structures

In Chapter 2, the Literature Review, all recent research work related to sign language recognition is discussed in detail, along with an overview of the existing applications currently available in the field. Chapter 3 presents the complete system design, covering both high-level and low-level design aspects, and includes comprehensive diagrams such as use case diagrams, entity-relationship diagrams, data flow diagrams, and sequence diagrams to illustrate the system architecture clearly. Chapter 4 focuses on the implementation phase, where all tools, frameworks, and technologies used in developing the system are described, and output screenshots are included to demonstrate the working application. Chapter 5 outlines the testing process, detailing the test cases created to validate the system’s functionality and reliability.

Chapter 2: Literature Review

2.1. Introduction

This chapter's purpose is to examine the research that is relevant to our work and how our work will be more enhanced and accurate than theirs. We will use specific training models and different methods to make our project better in every aspect of Continuous Sign Language Translation. We will keep in check that Hands that Speak will be checked frequently to keep it updated for new features so that new users will have it much easier to use. To organize our project features more accurately we will be creating an Excel table and in the final part we will try to summarize our work.

2.2. Training Models

In this field everybody uses different training models based on their needs and how it will affect their project more accurately, sustainability, time efficiency but some of the most prominent features and models used in this Continuous Sign Language Translations are as follows.

Convolutional Neural Networks (CNN's)

Convolutional Neural Networks are very prominent and widely used in the field of Sign Language Translation because it can extract features from video frames in real time making them effective in detecting hand gestures, hand movements, facial expressions, and body movements. **[8].** CNN provides the basic feature maps necessary for processing with using Recurrent Neural Network (RNN) and Transformers. Table 2.1 shows pros and cons of Convolutional Neural Networks**.**

Table 2. 1: Convolutional Neural Networks

Advantages	Disadvantages
Recognize every minor detail which includes hand movements, hand gestures, which is important for sign language.	Focuses on specific features and need to collaborate with other models for context in Continuous Sign Language Translation.

Adjusted in a way that it can handle high resolution video inputs with such improved precision.	Financially very expensive to analyse for large continuous video datasets that downgrade for CNN. **[16].**
Can comprehend differences in lighting, background, and the user signer with proper training.	Vulnerable for overfitting with the small and unbalanced with Continuous Sign Language datasets.
It can also work smoothly with other models such as Long Short-Term Memory (LSTM) for prediction.	Requires transfer learning from simple datasets, which are in risk of not detecting sign features accurately.
It can work in single model but can also collaborate with LSTM and Transformers which are effective.	Will find it difficult to capture indistinct movements, such as spelling on fingertips without proper architectural updating.

Transformers

In Continuous Sign Language Translation (CSLT) Transformer works on the mechanism of capturing dependencies and long-term sequences of real time video frames to certain context. **[5].** By processing the entire input sequence at once, Transformers model both different features like hand gesture, facial expressions and short-term dependencies like movement patterns very accurately. This allows the model to learn complex relationships between gestures with time and make meaningful sentences with textual representations. Transformers are advanced at handling the nature of sequence of sign language and can be integrated with vision models to update the understanding of continuous sign language. Table 2.2 shows pros and cons of transformers.

Table 2. 2: Transformers

Advantages	Disadvantages
Model specially works in long-term dependencies in sign language sequences important for continuous signing.	It demands significant computational resources especially for large video sequences. This is a fact when processing large scale of data.
Processes the input sequences concurrently, reducing time computation compared to other models.	It needs large and diverse datasets to prevent overfitting which guarantees the effective generalization in various situations.
Understand large number of datasets making it suitable for handling diverse sign language variations.	It may affect performance while dealing with incomplete sign language data or other problem which will effect on accuracy while interpreting.
Can be combined with other models like CNNs for feature extraction to handle both positional and momentary information effectively. **[7], [8], [13], [16]** (original Swin Transformer, ResNet, Attention paper)	Transformers will need extra features such as temporal attention for detailed motions in CSLT.
Very reliable in high accuracy with tasks in sequence including translation and captioning.	Transformers with long sequences will require a lot more memory which makes it difficult to run CSLT applications especially on the device that have limited number of features.

DINO:

Distilled Knowledge Transformers is a self-learning model that focuses on learning the visual representations without using labelled data. In the line of Continuous Sign Language Translation DINO works by learning useful visual features from video frames through distinctive learning. **[13].** It allows the model to capture important sign language features, such as hand shapes, body posture, and facial expressions. By training with this unsupervised

method, DINO creates representations that can help in recognizing signs and translating them into contextual texts. **[8].** DINO's ability to learn high-quality visual representations without requiring manual annotation. Table 2.3 shows pros and cons of Dino.

Table 2. 3: DINO

Advantages	Disadvantages
DINO do not need labelled data which is perfect for CSLT because annotated sign language data is rare and costly to obtain. **[16].**	DINO learns spatial features and will require extra models like RNN to understand the movement in continuous sign language.
It learns important and different visual features that shows difficult hand gestures and movements.	It can have problem with simultaneous gestures in continuous sign language. This needs extra methods to separate the gestures.
DINO can be trained on unmarked and large datasets helping it adapt to different sign languages. **[7].**	The self-supervised learning process can be costly. It needs a lot of processing power to work accurately.
The learned features can be used for other tasks like gesture recognition and this improves the efficiency of the CSLT system.	It is good at learning visual features, but it might confuse signs that look similar but have different meanings.
DINO picks up little and important details of sign language making it more accurate in gestures to text translation.	DINO is hard to adjust with other part of CSLT because it focuses on visual inputs and not on combining data.

DINOv2

DINOv2 Distilled Knowledge from Transformer v2 is an advanced self-supervised model that is improved from original DINO by providing more robust and scalable visual representations. In Continuous Sign Language Translation, it works by using a learning framework to clarify high-quality visual features from unannotated video data. It utilizes a vision transformer (ViT) backbone to process video frames by learning meaningful representations of hand shapes,

facial expressions, and body movements without the need for labelled data. It also captures both little details and high-level features from video sequences, making it suitable for recognizing complex sign language gestures. Table 2.4 shows pros and cons of DinoV2.

Table 2. 4: DINO v2

Advantages	Disadvantages
It does not require labelled data which is helpful for CSLT also benefits when getting large datasets.	It cannot handle time-based relationship. It will require other models to enhance time-based like Transformers.
It provides enhanced features that capture every detailed sign language gestures. This helps improve the accuracy of gesture recognition.	It can have trouble with consistently gestures in continuous sign language and might require help from other models.
DINOv2 detects both small details and context of gestures helping recognize subtle hand movements and facial expressions.	Training it on large datasets takes a lot of time and computing power which makes it harder to use in real-time CSLT.
It can work on unmarked large datasets improving its ability to work with different sign language and also works on new datasets.	It excels at distinguishing general visual features but struggle with similar-looking signs in CSLT. Context and subtle movement variations can pose threat.
It can also perform different tasks like action recognition it just need to require some feature adjustments.	It is prepared for videos which can make it harder for other CSLT formats or different sources to work.

Swin Transformer

Swin Transformer, or Shifted Window Transformer, is a hierarchical vision transformer architecture designed to efficiently process visual data with reduced computational cost. In the context of Continuous Sign Language Translation (CSLT), Swin Transformer works by partitioning video frames into local non-overlapping windows and computing self-attention

within these windows. By shifting the windows between layers, it allows information exchange across different regions of the frame, capturing both local and global context. This makes Swin Transformer particularly effective for recognizing spatially complex gestures and subtle facial expressions that are essential in sign language. Table 2.5 summarizes the key advantages and disadvantages of Swin Transformer in CSLT.

Table 2. 5: Swin Transformer

Advantages	Disadvantages
Efficient hierarchical design reduces computational cost while maintaining accuracy.	Primarily designed for spatial relationships; may require additional models to handle temporal dynamics in CSLT.
Shifted windows capture both local details and broader context, improving gesture recognition.	Larger window sizes may miss very fine-grained details in small gestures.
Flexible to different image sizes and resolutions, suitable for diverse sign language datasets.	Requires complex tuning and hyperparameter selection for best performance.
Pretrained models are widely available, helping achieve good results on smaller PSL datasets.	Processing long video sequences can still be resource-intensive.

SignFormer

SignFormer is a transformer-based architecture specifically designed for Continuous Sign Language Translation (CSLT). Unlike general-purpose vision transformers, SignFormer incorporates temporal modelling directly, making it capable of understanding the sequential nature of sign language gestures. It processes video frames as sequences and captures both spatial features, such as hand shapes and facial expressions, and temporal dependencies that occur over time. This combination helps the model recognize continuous signing, including coarticulation and subtle transitions between gestures. By using attention mechanisms,

SignFormer effectively focuses on important parts of the input sequence, improving translation accuracy. Table 2.6 highlights the main advantages and disadvantages of SignFormer in CSLT applications.

Table 2. 6: SignFormer

Advantages	Disadvantages
Directly models temporal dependencies, making it effective for continuous sequence	Requires large annotated datasets for best performance.
Attention mechanism helps focus on critical frames and subtle transitions in gestures.	Computationally expensive; real-time performance may require optimization.
Designed specifically for CSLT, leading to higher accuracy in translation tasks.	Training is complex due to the need to balance spatial and temporal attention
Handles articulation and sequential context, improving recognition of connected signs.	May overfit if trained on small or less diverse sign language datasets.

PGVT (Pose Guided Vision Transformer)

PGViT, or Pose Guided Vision Transformer, is a model that integrates human pose information with transformer- based architectures to improve sign language recognition and translation tasks. For Continuous Sign Language Translation, PGViT utilizes pose key points (such as hand joints and facial landmarks) alongside visual features to better capture the dynamic aspects of gestures. This dual input allows the model to focus on relevant motion and subtle hand movements that define sign language. By combining pose guidance with visual transformers, PGViT can effectively distinguish between similar signs and handle fast or overlapping gestures.

Table 2. 7: PGVT

Advantages	Disadvantages
Combines pose information and visual features, improving recognition of dynamic hand movements.	Requires reliable pose estimation; errors in pose detection can affect performance.
Better handles sequential dependencies, supporting real-time CSLT.	Pose detection increases computational complexity and may slow down processing.
Distinguishes subtle sign differences that look similar visually but differ in motion.	Needs high-quality video input to detect accurate poses.
Adaptable to different signers and variations in signing style.	Training requires additional annotated pose data alongside videos.

2.2.1 Terminologies

- **Sign language**: A language that uses gestures, facial expressions and body movements to convey message.
- **Continuous Sign Language Translation (CSLT):** Translating sign language videos to written or spoken words using sequence of gestures or expressions.
- **Vision transformer:** A deep learning model that divides images to patches and process them like a word.
- **Feature extraction:** Process of converting data into such representations that are used in learning models.

- **Spatial and Temporal features:** Spatial features are about position, shape and structure of hand in our case as we are dealing with sign language videos.

- While Temporal features are about how are these features evolve over time in sequence of frames.
- **Frame segmentation:** We divide video into region of interest (ROI) for focusing on analysis.

- **Sign Segmentation:** It is about finding the boundaries or gaps between each sign to improve translation.

2.2.2 Categorization of Existing Techniques

To classify the given techniques or models in CSLT, models can be categorized in different roles. CNNs outperforms at recognizing gestures and extracting different features, making them exceptional for tasks like hand shape identification.

Transformers are better suited for dynamic sequences, capturing motion and context through their attention mechanisms. Self-supervised models like DINO further updates the CSLT by enabling feature learning without extensive labelled data.

1. Convolutional Neural Networks (CNN)

2. Transformers

3. DINO

4. DINOv2

- **Main Features**: Main features for all those given models are under as follows:
- **Convolutional Layers:** Detects the spatial features from input images or videos using convolutional frames.
- **Pooling Layers:** Compress the feature maps to reduce the computational costs while having the important information.

- **Translation:** Maintain robustness to translates the input images or videos to ensure feature extraction.
- **Layers:** Convolutional layer and pooling layer work together to perform and classification tasks.

- **Positional Encoding**: Gives information about the different position of elements in the sequence and enabling the model to process sequential data without relying on recurrent structures.

- **Parallelization:** Detects all elements of the input sequence synchronously improving training efficiency compared to sequential models like RNNs.

- **Self-Supervised Learning:** Learns visual gestures without any labelled data using different learning to tell between similar and other data Points.

- **Transformer Based:** Takes a Vision Transformer from images or video frames making it more accurate for processing complex visual information.

- **Self-Attention:** Learn by the model to gain the importance of different parts of the input sequence like capturing long-range dependencies and contextual relationships.
- **Learning Strategy:** Student models learn from teacher models improving representation quality and make it more efficient learning of visual features.
- **Enhanced Self Supervised:** Improved learning that requires high quality representations from large undetected datasets.

- **Adaptability:** It can make work different input sizes like images and video frames by putting multiple layers and filters.

2.2.3 Limitations

There are different limitations and various challenges that we need to consider.

- **Feature specificity**: CNN focuses on spatial features but requires integration with other models for temporal features.
- **Overfitting:** On a small dataset there is a high probability that our model might over fit.

- **Computational cost: S**elf-supervised learning and training on large dataset can be computationally expensive.
- **Complex gestures:** Learning complex patterns requires additional methods or models to be used.
- **Real-time Challenges:** Models may face difficulty in real-time processing due to high computational demands. Table 2.8 shows real-time challenges that we faced.

Table 2. 8: Techniques in CSLT

Category	**Technique**	**Primary Role**	**Strengths**	**Challenges**
Static Recognition	CNNs (e.g., ResNet, VGG)	Identifying static gestures or poses	High accuracy in spatial feature extraction	Limited understanding of motion dynamics
Dynamic Sequences	Transformers (e.g., ViT, DINO)	Understanding motion and context	Captures temporal and contextual nuances	Requires high computational resources
Self-Supervised	DINO, BYOL	Pretraining for low resource datasets	No need for extensive labelled data	Sensitive to input inconsistencies

2.3. Proposed Improvements

Model selection: We chose vision transformers to better extract the temporal features as it will better handle dependencies on combining spatial and temporal features effectively.

Integration Techniques: We will combine a vision transformer with the Resnet model and we will also use multilingual translators to enhance the feature extraction and translation accuracy. [24] (SignFormer paper)

Dataset: We will be using different techniques like oversampling, data augmentation, pre-processing of dataset, and many other for better generalization.

2.4. Summary

Chapter 2 presents a comprehensive literature review focusing on recent advancements and research in sign language recognition, with special emphasis on techniques applicable to Pakistani Sign Language (PSL). It begins by discussing foundational training models, including convolutional neural networks (CNNs), transformers, DINO, and DINOv2, highlighting their role in computer vision and sequential data processing. The chapter then explores key terminologies and categorizes existing techniques used in continuous sign language translation (CSLT), identifying their advantages and limitations. Additionally, it examines related local and international applications, such as PSL datasets, real-time recognition apps, and transformer-based frameworks like SignFormer and Swin Transformer. Finally, the chapter proposes improvements over these existing methods by incorporating hybrid deep learning models and attention mechanisms to enhance recognition accuracy and adaptability to diverse PSL gestures.

Chapter 3: System Design

3.1 Introduction

The system design is one of the most important phases in software development because it contains very important information about how we are going to implement our work. In the making of a Continuous Sign Language-Translation we will mainly focus on creating an app that can convert sign language gestures into text or speech in real time accurately. It works by taking video inputs to identify important features like hand movements, facial expressions and hand gestures. We will use tools like CNNs with the help of LSTM which are used to detect these details, while other models like Transformers and DINO will help us to understand the flow of gestures and movements accurately with time.

These parts work together to recognize and translate signs in a continuous sequence. To improve the accuracy of the system, it includes methods to detect and recognize where one sign or sentence starts and ends. It also looks at additional details, like body movements and facial expressions to make translations more accurate. The app is designed to be quick and precise so it can work in real-time even on mobile devices with the help of powerful tools and processing. Continuous Sign Language Translation app will try to work and make communication easier between people who cannot speak and listen and those who can speak and listen.

3.1.1. Purpose

The main purpose of this application is to develop a system that uses Continuous Sign Language Recognition to take a video sequence as input, processes it in near to real time and analyse the sequence of each input frame to generate an output of collection of words in correct and meaningful sequence.

3.1.2 System Overview

The Continuous Sign Language Translation application will be designed to help in the communication between people who cannot speak and listen and use sign language and people who can speak and listen. This application will use advanced tools to handle the complex scenarios of sign language such as recognizing hand shapes, facial expressions, hand movements and body movements while also understanding the context and different gestures

in a sequence in near to real time. CNNs capture visual representations and on other hand Transformers process how these gestures fit together in near to real time. With the help of these technologies, we can translate sign language into clear text more accurately. Basic task of this application is to derive and extract frames from a video stream, then process group of frames to detect pauses and translate the gesture into a word afterwards using combination of different transformers. Then the system will check the context of the output word and if the output word is according to the context, it will accept the output. With all these advancements in technology the application faces few obstacles. One of the most prominent issues is the need for large and diverse datasets to train the models accurately and effectively. Unfortunately, some sign language datasets are often limited which challenges the system's ability to handle and work on different signing styles and variations because different regions use different signing gestures. Moreover, the system needs much more computational power making it very challenging to work on mobile devices or in real-time conditions where speed and efficiency are very crucial.

Expected Goals of the System

- We will try to enhance our project to Near to real time processing
- Our main priority is to get better accuracy.
- We will try to make this project for Language Localization.
- We will try to get our model train in a way that it can do frame extraction, and gesture detection, that can act independently.

3.1.3. Design Map

High level and low level diagrams are included in this section. The high level diagram consists of database structure and system architectures whereas low level diagram consists of comprises of unified modelling diagrams. The low level diagrams also consists of sequence diagram, entity relation diagram, and data flow diagram.

3.2. Design Consideration

Continuous Sign Language Translation, a few considerations must be addressed to make sure the system is accurate and user-friendly. These include ensuring gesture recognition accuracy through high-quality input devices like cameras. Environmental factors like lighting and background noise must be in check because they can affect recognition performance. The system will be designed with privacy especially when dealing with sensitive visual data and offer error with mechanisms for user feedback to improve accuracy over time. So, it can work effectively.

3.2.1. Assumptions

- It is assumed that the system will have access to good cameras to capture clear and detailed gesture data.
- The application assumes the capability to process gestures with minimal errors to communicate smoothly.
- The application might struggle while translating context-dependent signs and complex sentence structures.

3.2.2. Constraints

- Gesture Translation demands high computational power, which may not be available on all devices.
- Gesture recognition accuracy is limited by environmental factors like lighting and background noise.
- The system is limited by predefined datasets and struggles to handle regional gestures or new, evolving signs.
- Capturing and processing sensitive visual data requires privacy laws and requires data protection mechanisms.
- If user input is slightly different from the correct and given sign the app may not be able to translate properly.
- There must be one person in input frame.

Limitations

- Mobile devices are compatible with our app Hands that Speak, to use our application the user will need a fast, and dependable internet connection. If internet is weak app will shut down by then and no translation will be shown.
- Our app supports a specific size of input videos if exceeded app will not accept input.

Method of Design

Our application will require a single user at one time the recommended design for our application should be planned. Our application should be clear as we present. As we research we can adapt to clearer and easier.

3.2.3 Design Structure

To ensure design for effectiveness a design approach must try to reach improvement of system design based on functions response.

3.2.4 Risks and Boundaries

It should be important to connect to Internet. Any data that did not stored will be lost if Internet connection is lost, and also application will have technical problems.

3.2.5 Volatile Risks

When creating an application few risks should be checked that are under as follows:

Security Issues:

While using the app the personal data of user should always be protected by its granted mail, name and password. Only the user will have the security password and mail password so the personal data or history cannot be breached by any other person.

3.2.6. Overview

Focus more on system specific breakdowns and try to tackle why it has happened. Also give the important details about the main duties of functions of the system.

3.3. System Architecture

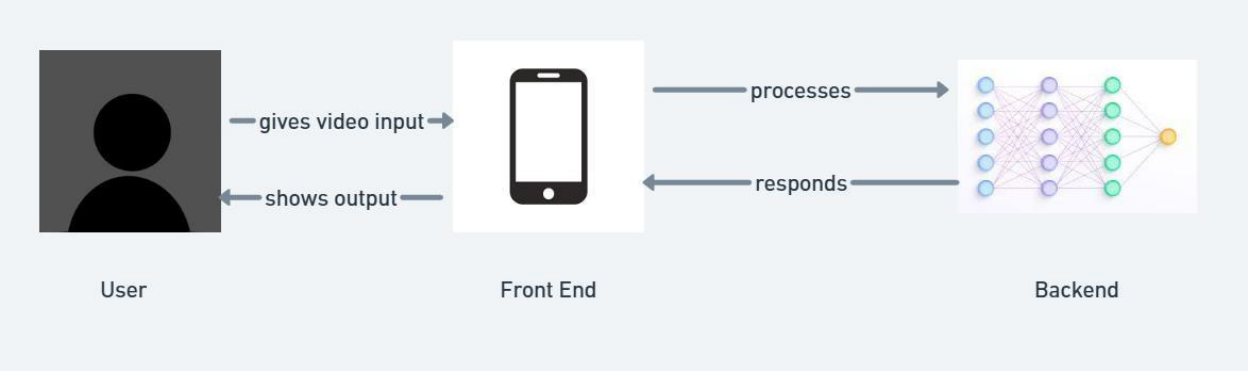


Fig. 3.1 System Architecture

3.4. High Level Design

These use case, ERD, and DFD diagrams below illustrate the top-level design of this application.

3.4.1. Use Case Diagram

The Continuous Sign-to-Text Translation system involves multiple actors and their functionalities to translate gestures into text. The actors and their attributes are:

Gesture Performer (Signer)

The person who performs the gestures that will be translated.

Attributes: Performer, Name, Role (e.g. Signer).

Interpreting System

The system is responsible for capturing, processing, and translating gestures. Attributes: Interpreter ID, Version, Capabilities.

Receiver

The person who receives the translation (text).

Attributes: Receiver ID, Name, Preferred Output Format.

4.Administrator

The person responsible for managing the system (e.g., updating models, managing datasets). Attributes: Admin ID, Name, Role, Permissions.

5.Translation Module

Manages translation input, processing, and output.

Attributes: Input Type (gesture), Output Type (text/voice).

Features

- **Gesture Capture:** Input gestures are captured with high-quality cameras.
- **Gesture Recognition:** Identify and classify gestures with advanced ML models.
- **Gesture to Text Translation:** Convert recognized gestures to text output.
- **Text Display:** Generate translated text in a human-readable format.

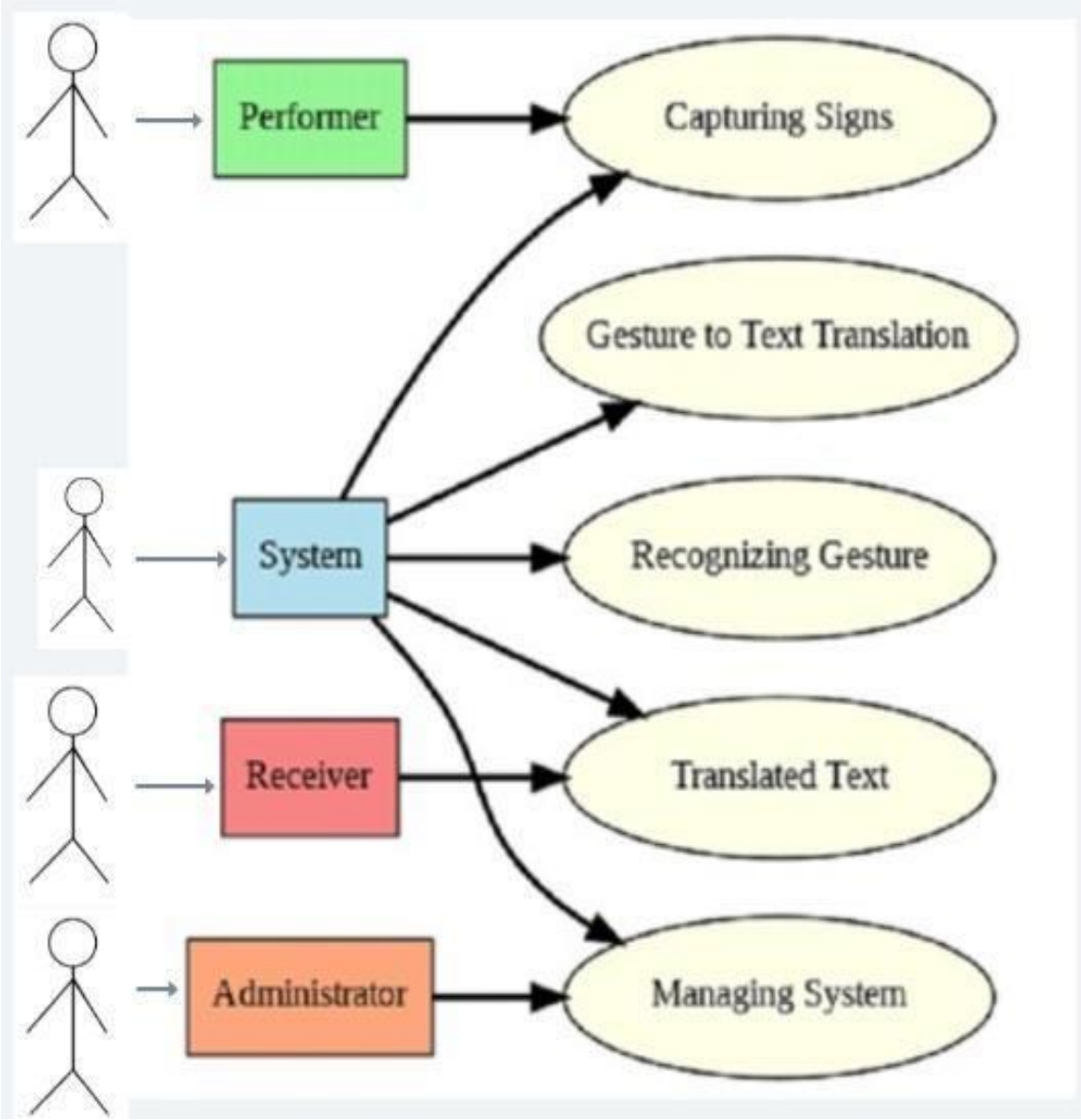


Fig. 3.2 Use Case Diagram

The following table shows use cases and their description.

Table 3. 1: Use Case

Actor	**Use Case**	**Description**
Performer	Capturing Signs	The performer provides gestures captured by the system.

Performer	Gesture to Text Translation	System translate gesture to the text.
System	Recognizing Gesture	System process and recognize input from performers.
System	Translated Text	The system provides translated text.
Receiver	Translated Text	Receiver receives text by system.

3.4.2. Full Functions on Use Case

Table 3. 2: Full Functions on Use Case

Function	**Description**	**Use Case**
Near Real-Time Translation	Converts sign language into text as the person signs.	Assists deaf individuals in conversations with hearing people by providing instant text output.
Educational Support	Provides textual output of signed lectures or lessons.	Supports students with hearing impairments in classrooms or online courses.
Customer Support Integration	Integrates into customer support systems to provide sign language users with text-based communication.	Enhances accessibility for deaf customers in service industries like banking or retail.

Healthcare Communication	Enables patients and healthcare providers to communicate via sign-to-text systems.	Facilitates doctor-patient interactions where sign language is needed.
Workplace Accessibility	Provides transcription of signed communication in workplace settings.	Allows deaf employees to participate in meetings or collaborative projects.
Public Service Announcements	Translates sign language announcements into text for broader accessibility.	Ensures accessibility during public announcements or emergencies.
Legal and Courtroom Applications	Offers accurate text transcription of sign language used during legal proceedings.	Assists in making legal environments more inclusive for deaf participants.
Media and Entertainment Accessibility	Provides real-time translation of sign language in media content like live shows or news broadcasts.	Makes entertainment and information accessible to a broader audience.
Social Media Integration	Integrates sign-to-text translation in social media platforms.	Allows deaf users to create content or interact with audiences who primarily use text.
Documentation and Record Keeping	Stores text translations of signed interactions for future reference.	Useful in medical, legal, or academic fields where maintaining records is essential.
Event Accessibility	Provides sign-to-text transcription during live events or webinars.	Enhances accessibility in conferences, webinars, or public talks.

Sign Language Learning Support	Offers text feedback for learners practicing sign language.	Aids in learning and improving sign language proficiency through direct text feedback.
Tourism and Public Interaction	Provides text translations of sign language in tourist areas or public services.	Improves communication for deaf individuals in unfamiliar environments.
Model Improvement	Collects data for improving AI models while ensuring user privacy and consent.	Enhances accuracy and performance of future sign language translation systems.

3.4.3. Context Diagram

A Context Diagram is a high-level data flow diagram (Level 0) that provides a big-picture view of a system.

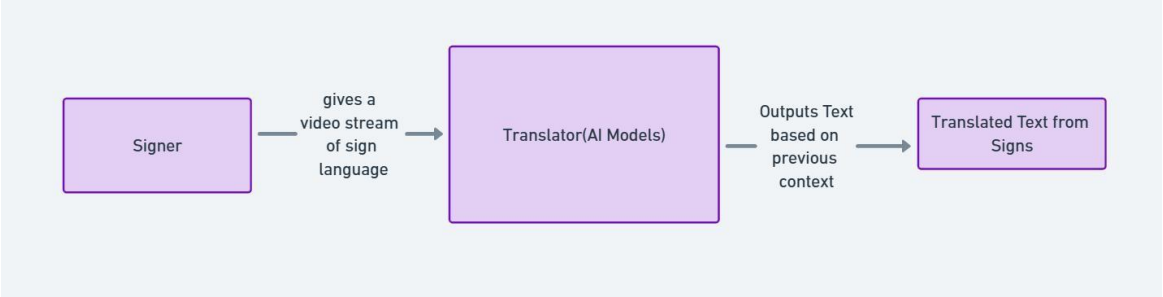


Fig. 3.4. Context Diagram

3.4.4. Data Flow Diagram (DFD)

Data Flow Diagram shows the flow of data within a system showing how the information is being processed, stored, and exchanged between processes. A **Data Flow Diagram (DFD)** is a graphical representation that depicts how data moves through a system. It shows the flow of information, the processes that transform the data, and the data storage elements. Figure 3.5 shows data flow diagram.

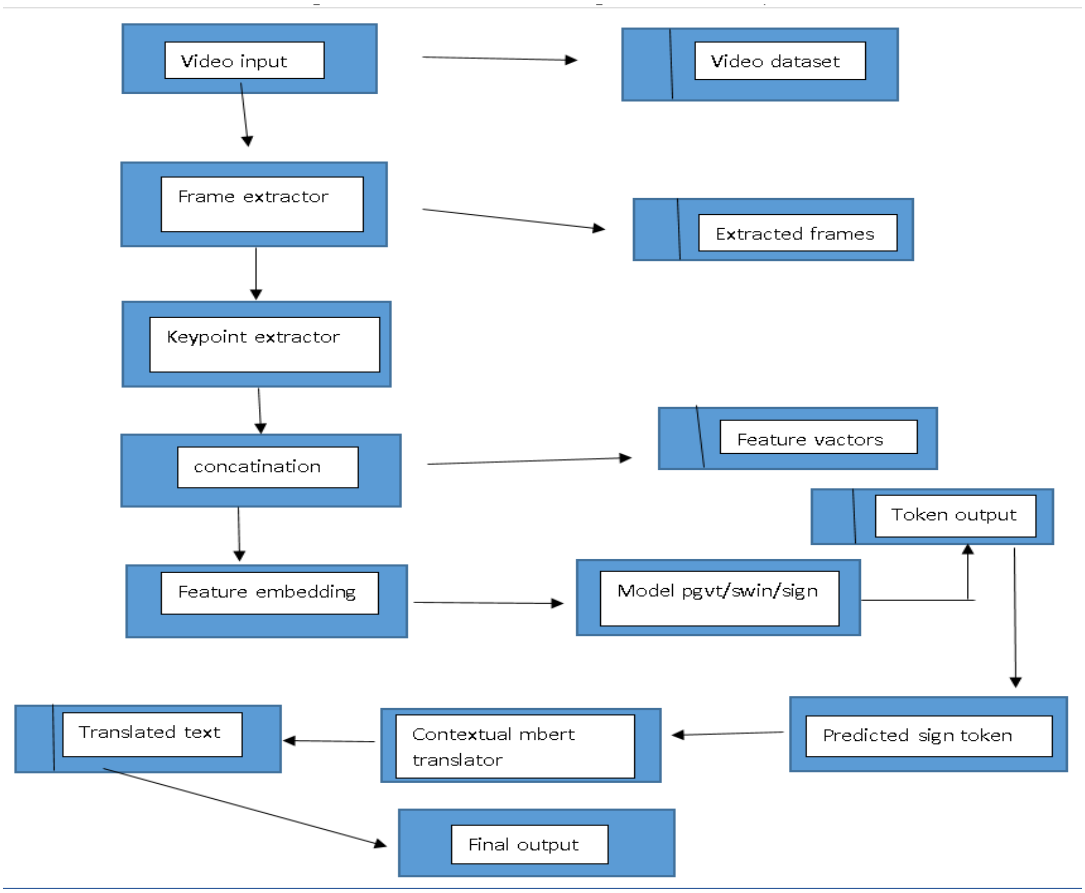


Fig. 3.5. Data Flow Diagram

3.5. Low Level Design

Activity and sequence diagrams are examples of low-level system diagrams used in this application.

3.5.1. Sequence Diagram

A sequence diagram for a Continuous Sign Language Translation (CSLT) system visually represents how different components of the system are related with time to achieve the translation process. The sequence diagram makes sure that a structured representation of the CSLT systems' behaviour and improving clarity with effective system development and communication. Following figures shows sequence diagrams for possible use cases. Figure 3.6 shows real-time translation process sequence diagram.

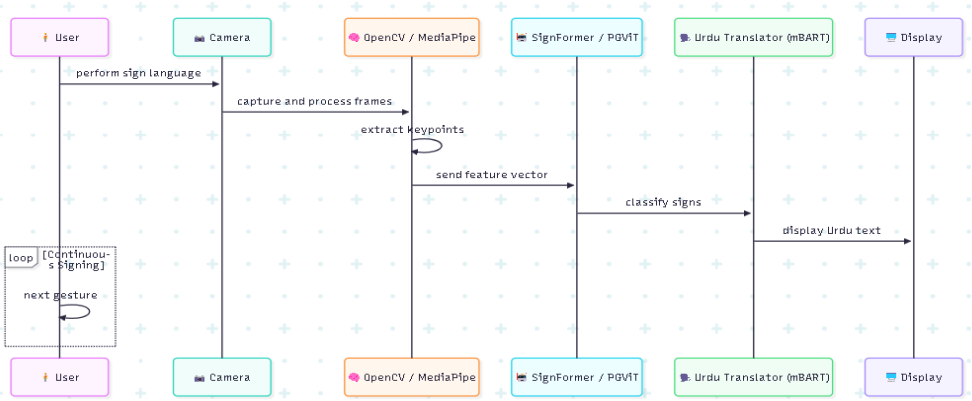


Fig 3.6 Real time translation

Figure 3.7 shows educational support process sequence diagram.

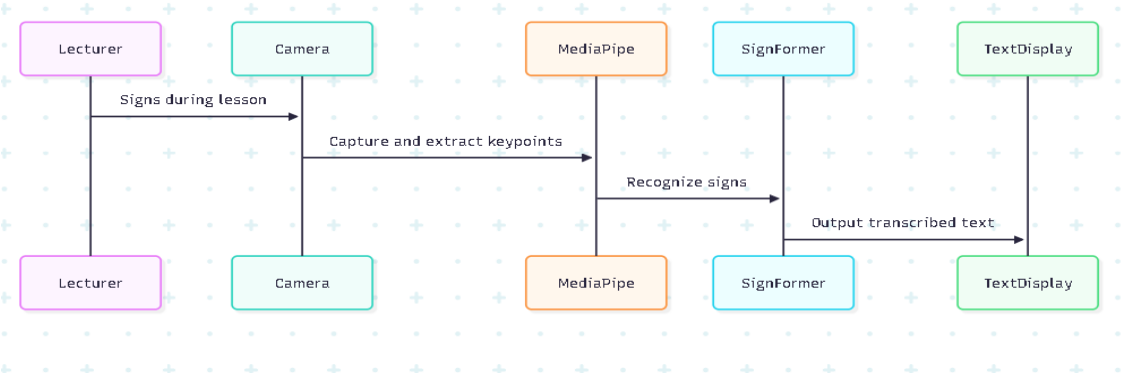


Fig 3.7 Educational support

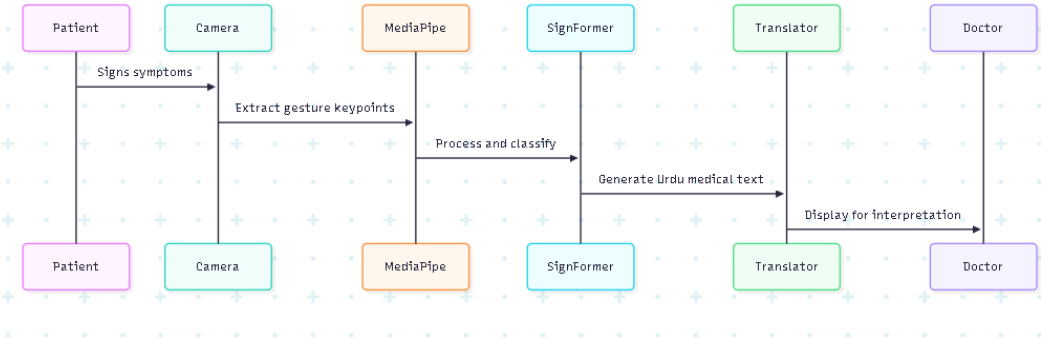


Fig 3.8 Health care support

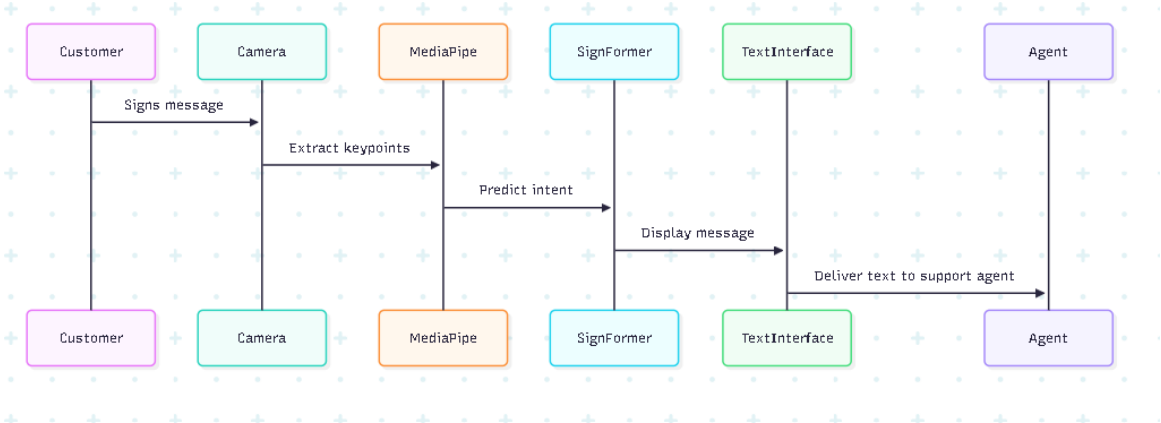


Fig. 3.9 Customer care support

3.5.2. Flow Chart

The Flow Chart for a Continuous Sign Language Translation (CSLT) system provides a structured visual representation of the flow of control and data through different activities involved in the sign-to-text translation process. Figure 3.10 shows activity diagram.

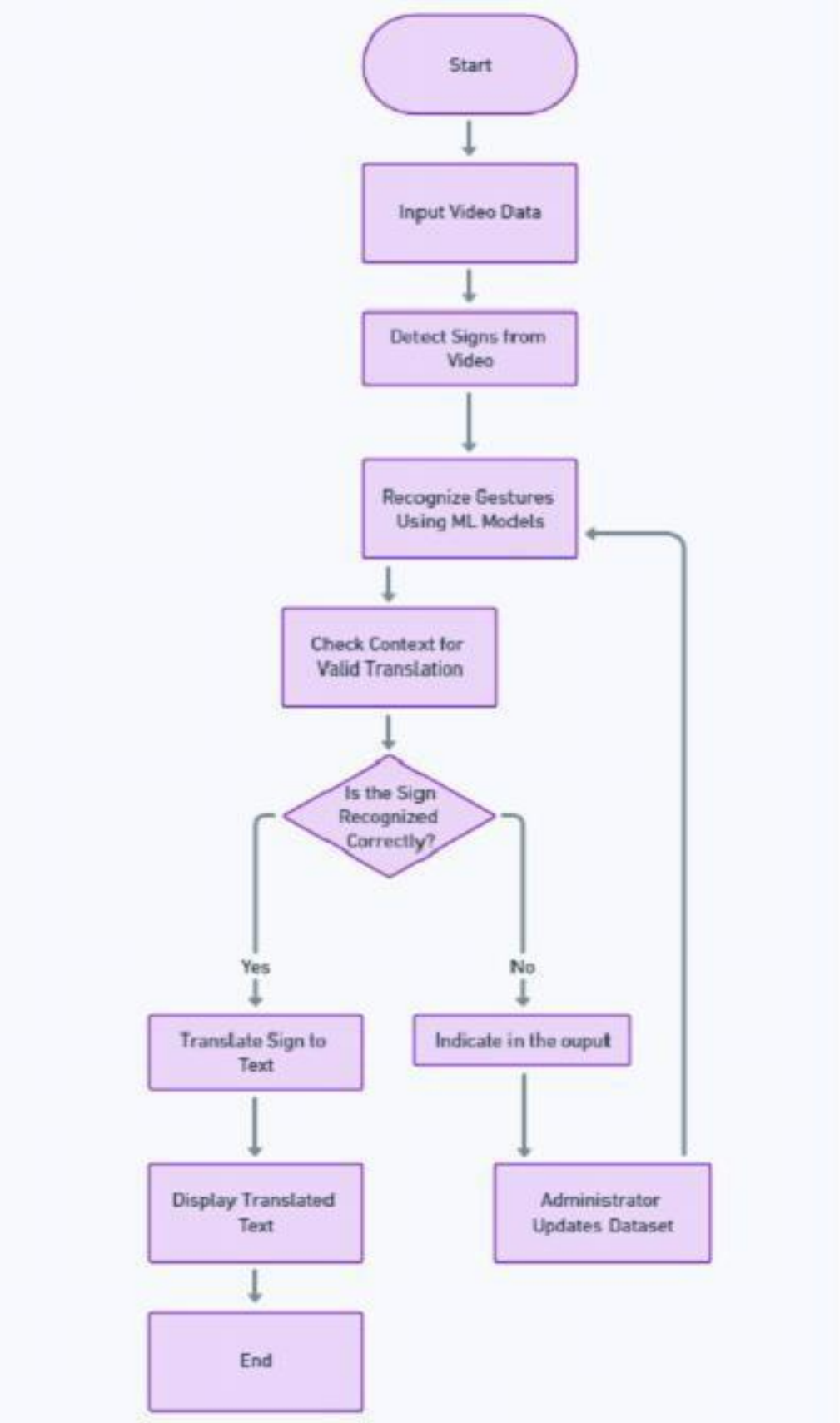


Fig. 3.10. Flow Chart

3.6. User Interface Design

This section offers user interface design discussions that supports the development of user interface displays.

3.6.1. Authentication Screens

Figure 3.11 and Figure 3.12 shows home screen and login screen of the user interface.

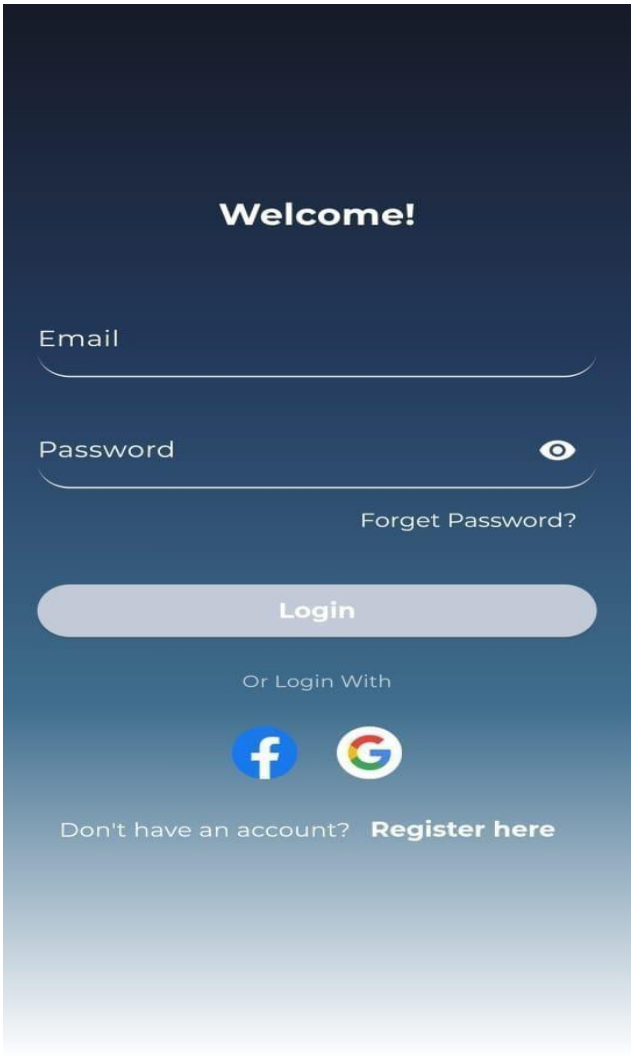


Figure 3. 11: Login Screen

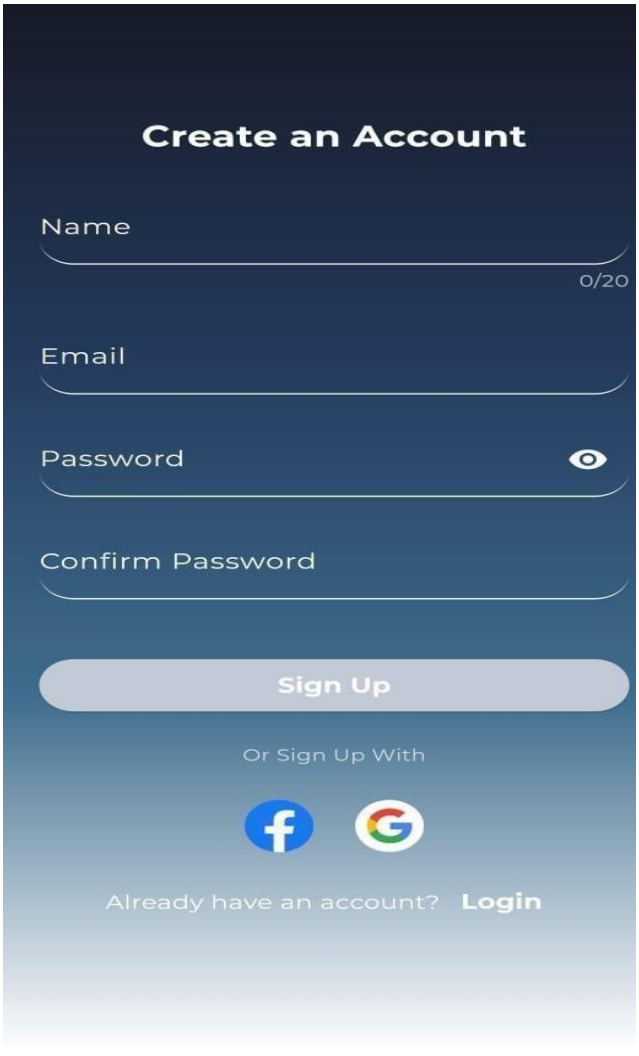


Figure 3. 12: Sign Up Screen

3.2.1. Home Page Screens

Figure 3.13(a) and Figure 3.13(b) show home screen.

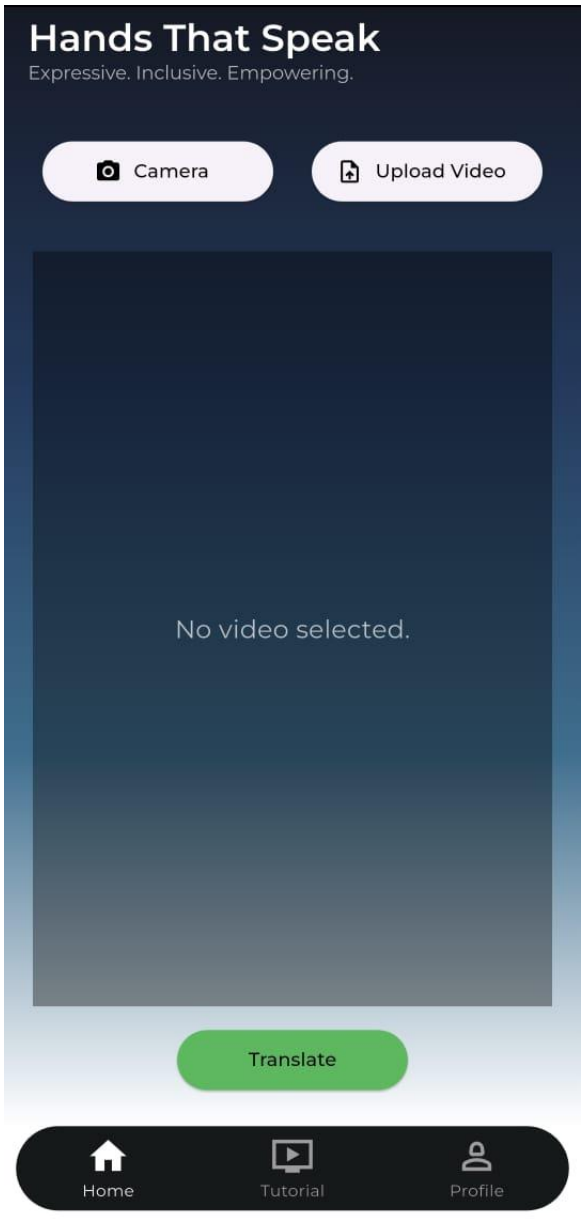


Figure 3. 13: Home Screen

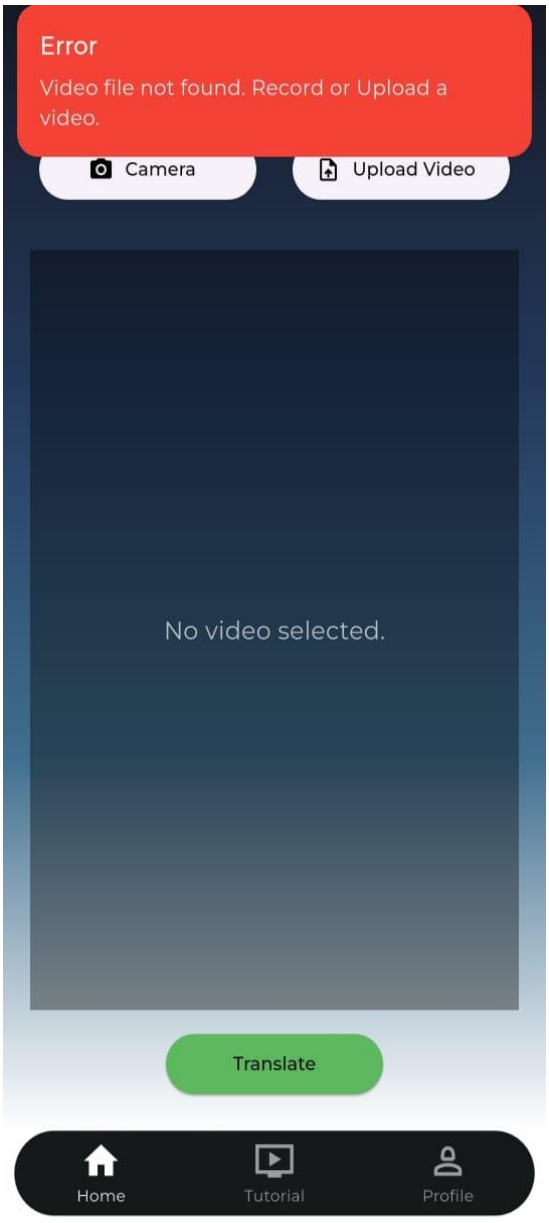


Figure 3. 14: Error

3.2.2. Tutorial Screens

Figure 3.14 and Figure 3.15 show the tutorial screens.

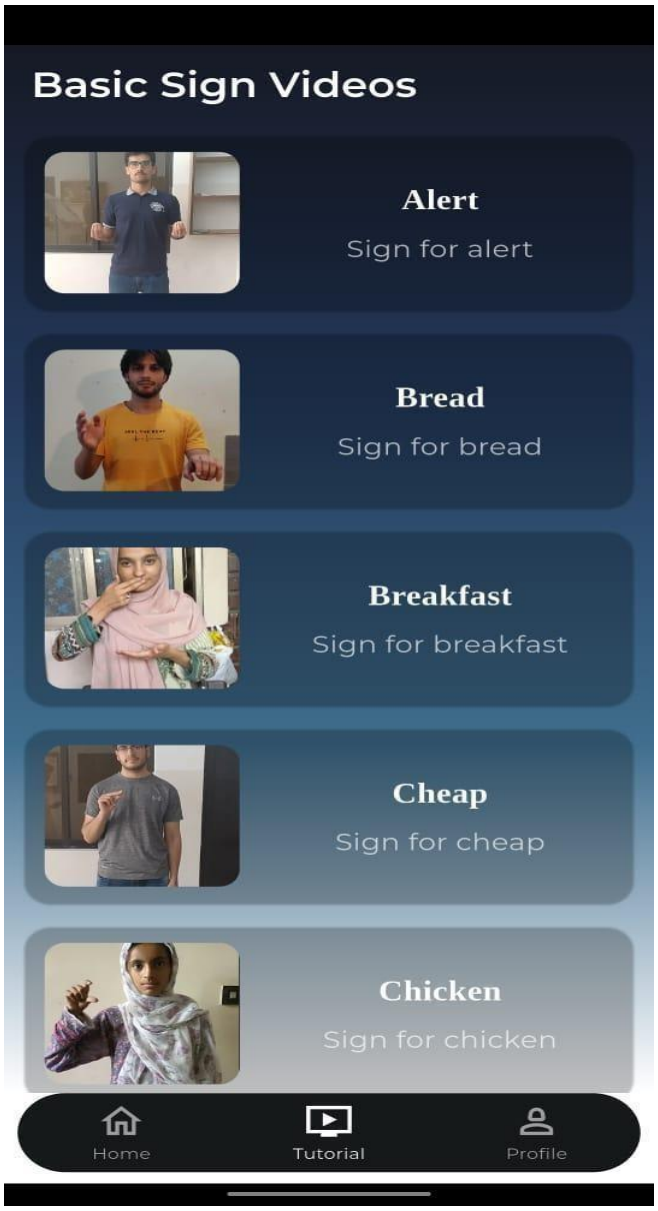


Figure 3. 15: Tutorial Screen

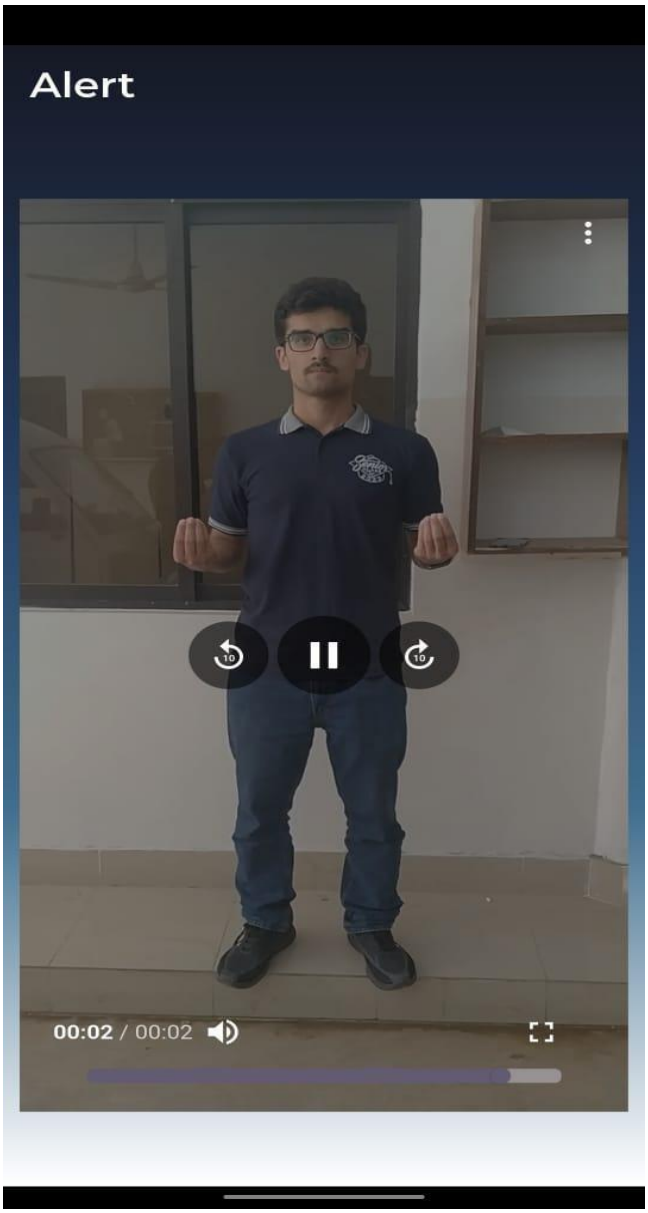


Figure 3. 16: Tutorial Video

3.2.3. Video Screens

Figure 3.16 and Figure 3.17 shows video recording and uploading screens.

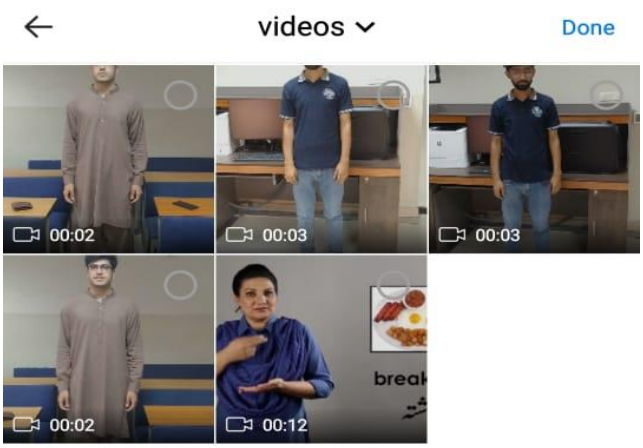


Figure 3. 17: Video Uploading Screen

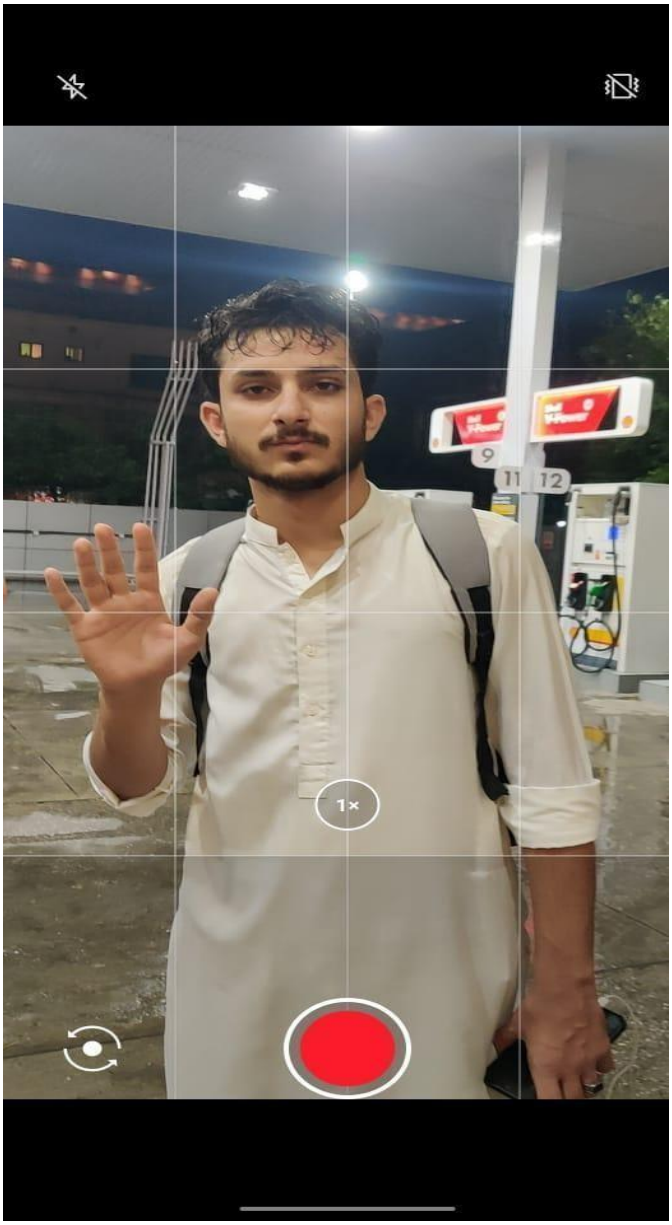


Figure 3. 18: Recording

3.2.4. Translation Screens

Figure 3.18 and Figure 3.18 shows video recording and uploading screens.

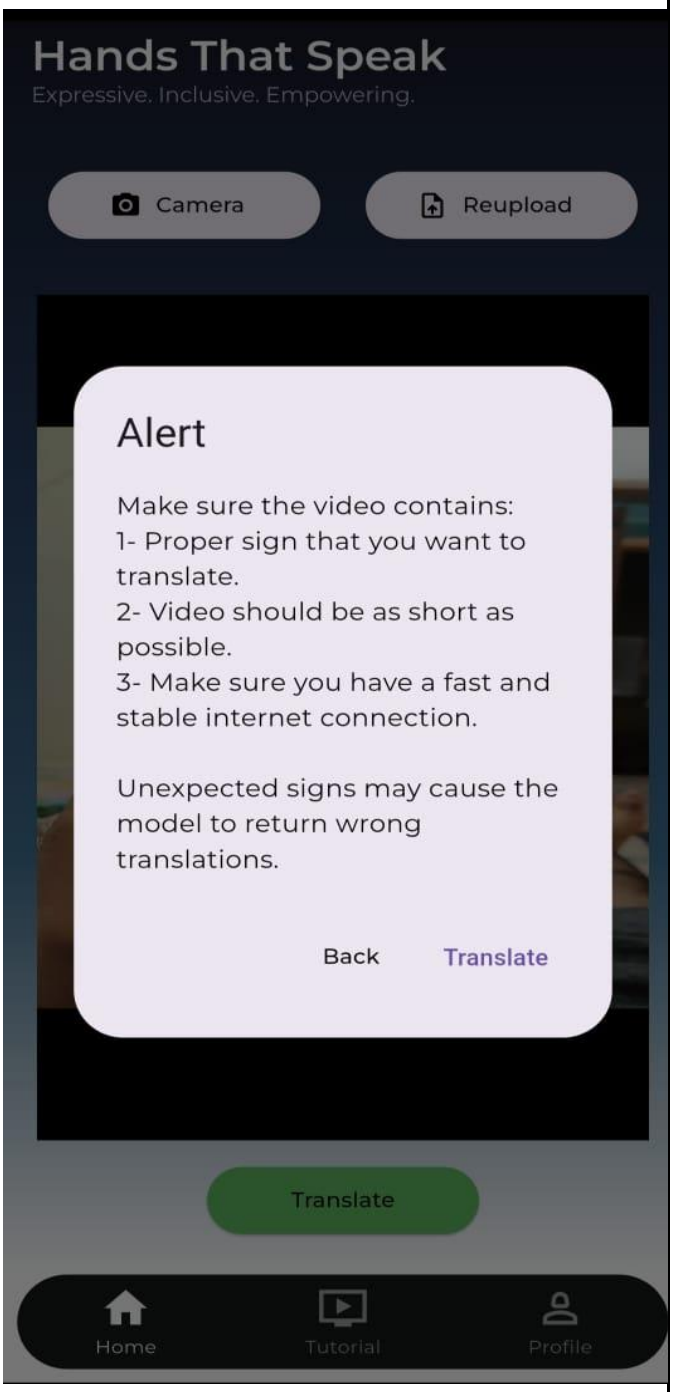


Figure 3. 19: Continue Alert

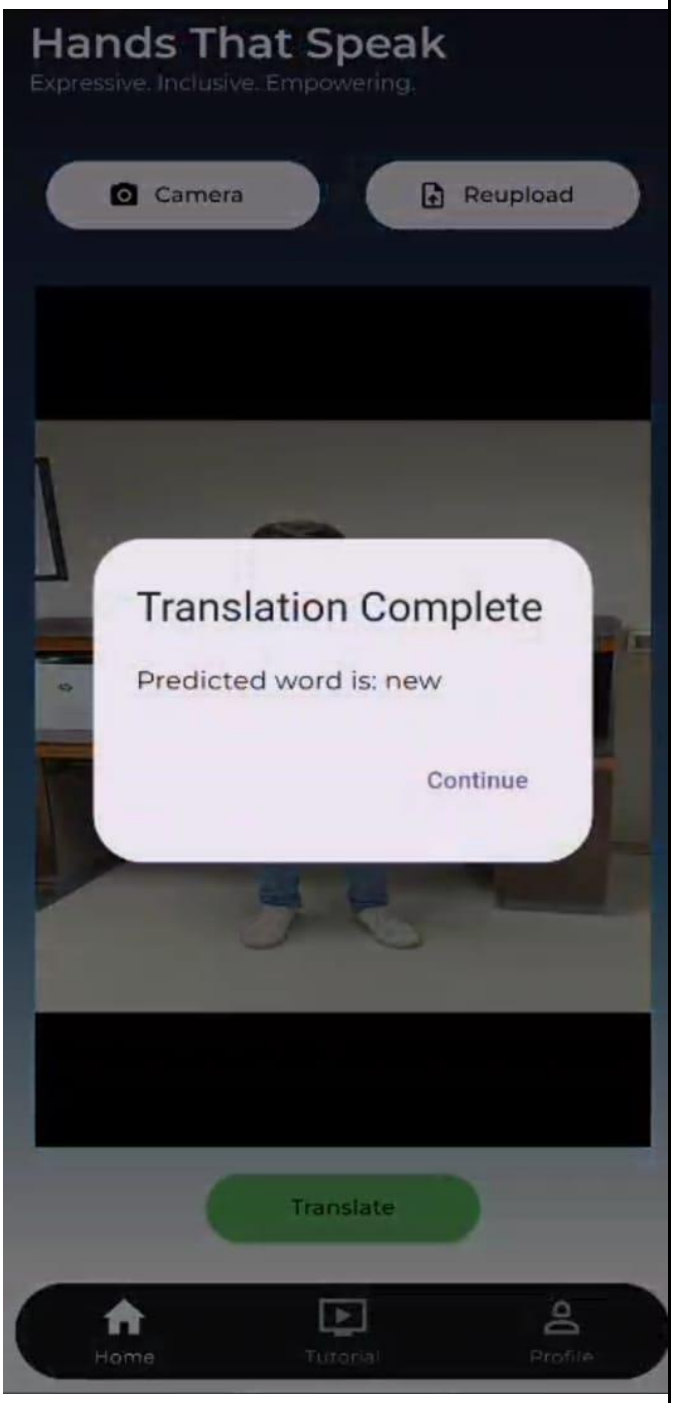


Figure 3 20: Output

3.7. Risks and Changing Boundaries

During the making of the Continuous Sign Language Translation (CSLT) system comes various risks. Main challenge is diverse datasets with limited availability, which sometimes fail to represent the full range of regional variations in sign languages. **[7], [8].** This can result in inaccuracies in translations, especially for underrepresented signing styles. Moreover, the complexity of continuous signing, including different gestures and subtle variations in hand movements, makes accurate recognition and translation very difficult. These issues can reduce the system's reliability and effectiveness, and translation.

As time passes and AI evolves with the boundaries of CSLT continue to advance with advancements in AI and machine learning. Transformations like self-supervised learning. Optimization offers the potential for more accurate and effective systems. But these changes also bring new demands, such as more powerful computational demands and also the challenge of adapting existing systems to combine advanced techniques. Keeping these in check, these evolving boundaries require continuous updating of models, improved data collection strategies that balance performance with accuracy. Managing these shifts is very important to make sure CSLT systems remain accurate, reliable, and practical.

3.8. Summary

We are making the Continuous Sign Language Translation (CSLT) application is made to help children in kindergarten who use sign language communication with those who cannot understand them. The application will work by using a camera to capture the user's hand gestures or hand movements. These gestures are then processed and turned into text. So, others can understand the sign user person. The application will be carefully tested and trained to make sure it works well in different circumstances such as recognizing fast hand gestures or working in dim light. A sequence diagram helps show how the system works from one step to another from when the user makes a gesture to when the translation appears on the screen, and all of this is happening at one time. The application also saves the users data so the user can read its history so it can be improved over time. Our main goal is making it easy to use and reliable. The CSLT system helps children and people who use sign language communicate more easily with everyone around them without less rest.

Chapter 4: Implementation

4.1 Discussion

The implementation of this research is based on the previous chapter design framework chapter 3. Important designs such as flowcharts and diagrams of workflow, data, and architecture were important in making the project clearer during making of our project. The project structure performs many components, such as data pre-processing, model training and testing and has used other models and methods to build this project.

4.1.1 Implementation Design

The main purpose of making the project is to translate the hand gestures or hand movements of video frames into text translation. This project involves use of deep learning models and other approaches like Transformers. The model architecture can be understood by diagrams, flowcharts that connect the relationship between datasets, pre-processing and training. These diagrams ensure an authentic workflow and how the system works.

4.1.2 Challenges Encountered

While Implementation few challenges occurred.

Dataset Problems: To make sure that the labelled datasets were balanced and ready to perform tasks was really time consuming.

Model Training: To train complex models for high accuracy and using oversampling was tiring.

Feature Extractions: It was a difficult process to extract the features and very difficult to manage time sessions.

4.1.3 Problem Solving

Challenges were approached with combinations of research, experimentations, and collaborations.

Training the whole model was difficult so it was divided into annotations to work efficiently and without complications.

Iterative testing the models and consulting with our supervisor to work with them accurately.

Dropped using Dino vision transformers because it was being complex and was not working up to our expectations.

4.2 Research Methodologies

The project followed multiple methodologies to complete its objective accurately and effectively.

4.2.1 Agile Research

The project was divided into some parts, like data collection, pre-processing and model training.

4.2.2 Iterative Method

Iterative methods were made based on results of experiments. Iterations were focused on improving model performance and based on accuracy. Pre-processing workflows are refined iteratively.

4.2.3 Testing and Validation-Driven Research

Experiments were designed to validate the performance of models like accuracy and Grad-Cam visualizations.

4.3 Implementation of Tools

Some of the tools were used to support the research process accurately.

4.3.1 Frameworks and Libraries

- **Tensor Flow:** For making and training the deep learning models.
- **Grad-CAM:** For implementing techniques and visualizing model predictions.

4.3.2 Development Tools

- **Google Collab**: We used it for Visualizations and experimentations.
- **Python (Pycharm):** It was used for writing and debugging code.
- **Flutter and android studio:** for user interface and design.

4.3.3 Libraries Tools

- **Pandas and NumPy:** It was used for pre-processing and organizations.
- **Matplotlib and Seaborn:** It was used for visualizing the result of experimentations and also the Grad-CAM outputs.
- **MediaPy:** We used MediaPy in order to work with and process video files.
- **CV2:** We used it for Images loading, and also images manipulation.
- **PCA:** It is used to reduce the dimensionality of features, so that will be able to present significant information and also make a small set of variables.

4.4 Implementation Features

There are specific technologies and algorithms used on this project that are under as follows.

4.4.1 Algorithms and Methods

Different algorithms and methods were used as part of the research project to get the accurate sign language and to make sure to get accurate results. The following techniques are under as follows.

Pre-processing Methods:

- **Frame Extraction:** In order to extract the frames from the videos we used a fixed size of (224x224) for Swin transformer because it does not work on bigger size.
- **Data Augmentation:** Synthetic data generation methods include methods like rotation, flipping and colour variations were applied to balance the dataset.
- **Concatenation:** Joined frames extracted from videos into a single sequence and then used these concatenated frames into our model.

Model Development:

- **Swin Video Transformer:** It is a multi-scale transformer that is used for efficient processing of high-resolution images which are used in classification.
- **Sign Former:** A video transformer used for context-based translation. We used initial layers to get our sign translated.
- **PGVT:** Pose guided vision transformer used for human movement translation and key point-based pose information as input.
- **Resnet 50:** It is a convolutional neural network (CNN) consisting of deep 50 layers.
- **Mbert Translator:** Multilingual Bert a deep learning model that translates the text to base on context.

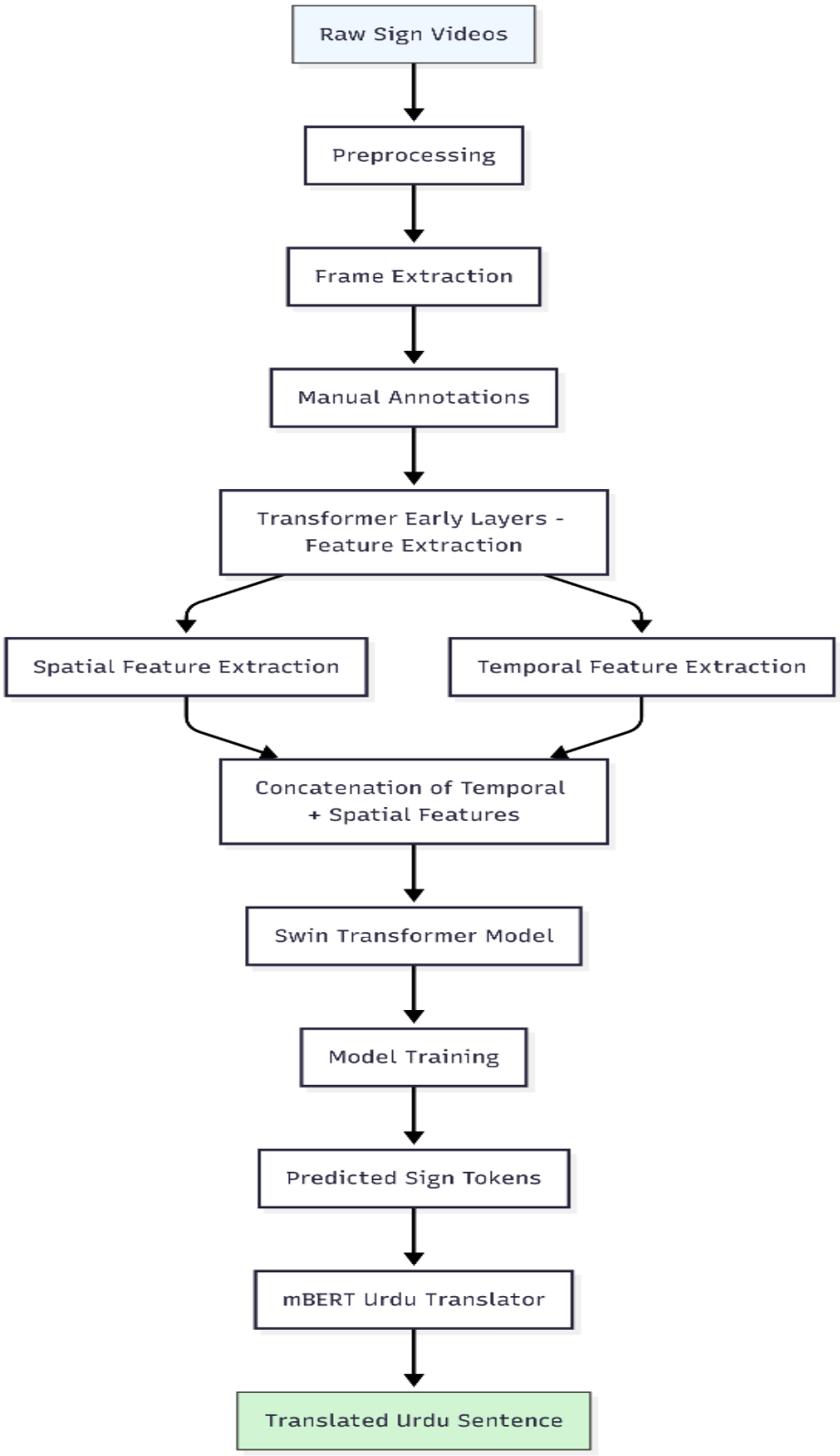


Fig 4.1. Design methodology

Explain ability Method:

- **Grad-CAM:** It is used to generate heat maps to visualize the regions of images.
- **T- SNE:** It is a technique used for data exploration and visualization of high dimensional data.
- **Inference Time:** It is a time for a model to process the data and make a prediction.

4.4.2 Datasets

Our Dataset includes multiple classes such as bad, surprise, stiff etc that contains sign language videos.

To improve the robustness and diversity of our dataset, we expanded beyond the initial 7 gesture classes by incorporating additional sign language videos sourced from the official Pakistan Sign Language (PSL) website**. [28], [30] (PSL dataset references***).* Specifically, we collected one representative video per new sign. Since the original dataset had limited samples per class, we applied a comprehensive data augmentation strategy including rotation, flipping, brightness variation, and noise addition to synthetically increase sample size and variability. After performing these augmentations, we concatenated the new data with our original dataset, ensuring class balance. This process was repeated for multiple iterations to refine the dataset further, improve generalization, and reduce overfitting. As a result, our updated dataset now reflects a more diverse set of sign variations and signer appearances, enabling our model to perform better in real-world conditions.

Techniques

To enhance model performance and address the limitations of a small dataset, we applied a series of data engineering techniques**.** These included:

Data Augmentation: Applied transformations like horizontal/vertical flipping, rotation, zoom, brightness adjustment, and noise injection to synthetically increase the dataset size. **[30], [26].** To artificially increase the diversity of sign language videos and improve model generalization, a variety of data augmentation techniques were applied to the raw video data. Table 4.1 shows augmentation techniques that were implemented both spatially and temporally.

Table 4. 1: Augmentation Techniques

Technique	Description
Crop	Random cropping to simulate zoomed-in views
Rot10	Rotation by +30 degrees
Resize0	Resize frame to 50% to simulate distant camera views
Resize150	Resize frame to 150% to simulate close-up views
Bw	Convert frames to black and white
Invert	Invert frame colours to simulate lighting shifts
Bright	Random brightness adjustment
slow	Decrease playback speed to simulate slower gestures

Class-wise Concatenation: After augmentation, the new video samples were carefully concatenated class-wise with the original data to ensure class balance and maintain label integrity.

Custom Dataset Curation: Additional videos were sourced from the Pakistan Sign Language (PSL) website. One representative video per sign was added for underrepresented classes.

Iterative Refinement: This process was repeated iteratively—augment → concatenate → retrain—allowing the model to generalize better across variations in signers, lighting, and backgrounds.

These pre-processing and augmentation techniques helped to combat overfitting, increase intra-class diversity, and make the model more robust for real-world deployment.

Variability

It consists of sign language videos of different individuals with different words to increase diversity of dataset.

Annotation Quality

All datasets were verified for accurate class labelling to maintain in training and evaluations.

4.4.3. Screenshots: [35], [36] (MobileNetV2, user study)

Labelled Frames



Fig. 4.2. Labelled Frame

Important Features (After Applying PCA)

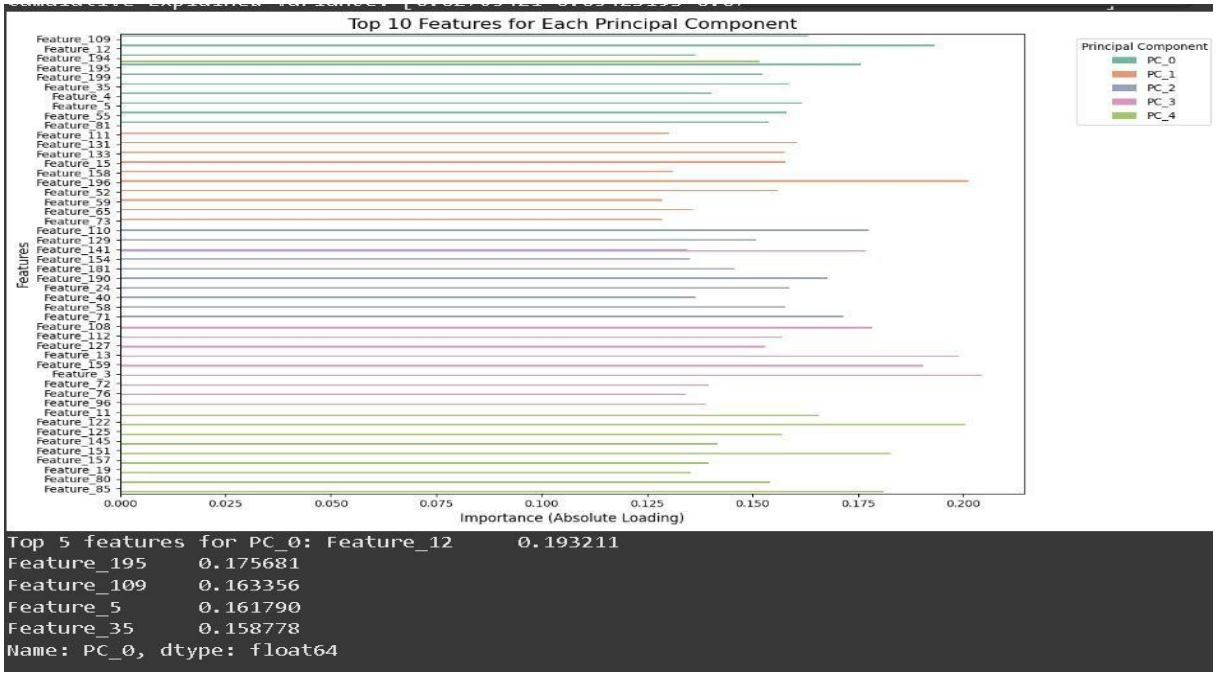


Fig. 4.3 Plotting Important Features

Hand Landmark



Fig. 4.4 Hand Landmark

4.5. Summary

In this chapter, we detailed the complete implementation pipeline for our continuous sign language recognition system. The process began with structured video pre-processing, where raw sign language videos were converted into frames at a consistent frame rate. A comprehensive set of data augmentation techniques including cropping, rotation, resizing, brightness adjustment, and speed variation was applied to enhance dataset diversity and reduce overfitting.

Key points were extracted from each frame using pose estimation tools, and were then normalized and concatenated to form temporal feature sequences suitable for model input. Both PGViT and SignFormer models were trained using this processed data. SignFormer, in particular, was chosen for its transformer-based sequence modelling capabilities tailored to sign recognition tasks.

Additional videos were sourced from the official Pakistan Sign Language (PSL) website to extend the dataset beyond the initial 7 classes. **[39], [40].**These new samples were integrated with the original dataset using iterative augmentation and retraining loops to enhance performance and class balance. The chapter also outlined consistency strategies, model input formatting, and key implementation steps that contributed to the robustness and accuracy of the system.

Chapter 5: Testing

5.1. Testing Techniques Employed for This Project

The testing phase of the Sign Language Translation project focused on ensuring the accuracy, robustness, and adaptability of our deep learning models across varied conditions. The system was designed to recognize sign language gestures and translate them into textual output using advanced neural networks. The following techniques were adopted to thoroughly evaluate the system's performance.

5.1.1. Pre-processing & Environment Testing

Greyscale Conversion: Testing confirmed that converting videos to greyscale led to a noticeable increase in accuracy compared to colour or black-and-white backgrounds.

Data Augmentation: Applied various transformations such as flipping, rotation, and zoom to artificially expand the dataset and reduce overfitting. **[31], [33].**

Background Sensitivity: Accuracy dropped significantly when gestures were recorded against pure black or white backgrounds due to loss of contrast.

5.1.2. Model Testing and Comparisons

CNN + RNN/LSTM

Our primary pipeline involved using CNNs for spatial feature extraction and RNNs/LSTMs for temporal sequence modelling. This model yielded consistent accuracy on trained videos.

Swin Transformer vs Sign former

Initial tests using the standard Swin Transformer led to memory overflow and system crashes due to high resource demand. The Sign Former variant was later used, enabling successful training with acceptable performance metrics.

MBERT Implementation

Unmodified mBERT was first used with Hindi gestures. Output was limited and not contextually refined.

Modified mBERT produced better contextual translations when applied to Urdu, enhancing semantic accuracy.

5.1.3. Training Strategy Evaluation

Initially, the model was trained on all gesture videos in one go, but this approach led to lower generalization and hardware instability. We transitioned to training on one video at a time, which stabilized memory usage and improved class-wise accuracy. The number of epochs was varied significantly to tune model convergence and avoid overfitting or underfitting.

5.1.4. Evaluation Metrics

Precision-Recall Matrices: Enabled analysis of false positives/negatives and class-wise balance.

Confusion Matrices: Helped identify which specific signs were most frequently misclassified.

F1 score: used in classification report especially when we want a balance between recall and precision.

5.1.5. Limitations and Observations

- The model is currently limited to recognizing only 7 different signs.
- Accuracy is heavily influenced by background contrast and lighting conditions.
- The best performance was observed in greyscale mode, suggesting improved generalization by reducing background noise.
- Real-time deployment remains a challenge due to model size and hardware constraints.

-

Table 5. 1: Test cases and their results.

Case ID	Scenario	Mapped Use Case(s)	Algorithm	Reason
TC01	Video Pre-processing	All Use Cases	PGViT / SignFormer	Pre-processing improves model performance across all scenarios by enhancing input video quality.
TC02	Model Training (CNN-RNN, all videos)	EducationalSupport, Public Announcements, Media Accessibility	Baseline CNN-RNN	General sign recognition needed in diverse lectures, broadcasts, and public info settings.
TC02	Model Training (per video)	Sign Language Learning Support, Documentation & Record Keeping	PGViT	Improves class-specific accuracy crucial for learners and for maintaining legal/medical records.
TC02	Swin Transformer (crashes)	Media Accessibility, Social Media, Event Accessibility	Swin Transformer	High-end transformer model fails under memory pressure; affects live & media-heavy use cases.
TC02	Swin Tiny	Near Real-Time Translation, Customer Support, Workplace Accessibility	Swin Tiny	The lighter model works well in real-time, low-resource settings where speed and uptime matter.
TC02	Language Model - mBERT (Urdu)	Tourism/Public Interaction, Educational Support	mBERT	Evaluates cross-language capability important for non-Urdu users or multilingual regions
TC02	Visualization (heatmaps, metrics)	Model Improvement	PGViT / SignFormer / Swin	Helps researchers/engineers fine-tune model behaviour for accuracy and performance.

5.3. Test Results

Table 5. 2: Test results

Metric	Observation
Average Accuracy	86% on greyscale inputs
Background Sensitivity	Accuracy drops by 15–20% in black/white backgrounds
Latency	200ms per prediction on CPU
Language Accuracy	Urdu mBERT output more reliable than Hindi
Stability	Swin Tiny model runs without crashing; full Swin Transformer failed

5.4. Summary

This chapter detailed the exhaustive testing procedures used to validate our sign language translation system. We experimented with multiple neural network architectures and pre-processing strategies to optimize model performance and stability. The CNN-RNN-LSTM approach, paired with mBERT for multilingual translation, proved effective in controlled environments, particularly in greyscale conditions.

While our current setup is limited to 7 gesture classes and is sensitive to certain backgrounds, future improvements will focus on expanding vocabulary, optimizing for real-time use, and enhancing background-invariant recognition. The rigorous use of visual metrics like heat maps and precision-recall matrices further confirmed the model's interpretability and performance.

Chapter 6: Data Analysis and Results

6.1. Methodology

The empirical study for this project was conducted to rigorously evaluate the performance of the proposed Sign Language Translation system under controlled and variable conditions. Our methodology followed a scientific experimental approach, ensuring reproducibility, clarity of objectives, and consistency across datasets and models. The study was segmented into multiple phases, including data pre-processing, model training, evaluation using standardized metrics, and comparative analysis. We employed baseline comparisons, parameter tuning, and multiple test cases to validate our results in diverse scenarios.

We conducted experiments on a curated dataset comprising continuous Urdu Sign Language sentences annotated in both Urdu and English. Each model is trained end-to-end, using tokenized spatial-temporal features obtained from raw video inputs. The token sequences are processed through the respective transformer encoder-decoder frameworks. The output is compared against ground truth translations.

All models were evaluated using 5-fold cross-validation, and results were averaged across test sets.

6.2. Empirical Study Structure

The empirical study was structured as follows:

- Data was pre-processed using greyscale conversion and augmentation to enhance training robustness.
- Models including CNN-RNN, CNN-LSTM, and mBERT variants were trained on the same dataset. **[34].**
- Swin Transformer and Swin Tiny were evaluated to understand the impact of model size and complexity.

Test cases were executed to capture real-world performance under varying background and lighting conditions. Evaluation was performed using multiple metrics including accuracy, precision, recall, and latency. Results were visualized using precision-recall matrices and heatmaps for interpretability.

6.3. Metrics for Result Comparison

To ensure objectivity and thorough evaluation, the following metrics were used for comparing model performance **[31], [32]:**

- **Accuracy:** Measures the proportion of correctly predicted signs over total samples.
- **Precision and Recall:** Evaluates model's performance per class to account for class imbalance.
- **F1 Score:** Harmonic mean of precision and recall, representing the balance between false positives and false.
- **Latency:** Measured as average prediction time in milliseconds per frame.
- **Confusion Matrix:** Visual tool for identifying class-wise misclassifications.
- **Model Size and Resource Footprint:** To assess the feasibility of real-time deployment.

6.4. Performance Evaluation

The performance of each model was empirically evaluated based on the test set. Swin Tiny outperformed the full Swin Transformer in stability and memory efficiency, though it slightly lagged in feature depth. CNN-RNN and CNN-LSTM combinations demonstrated strong accuracy, particularly when trained with greyscale data and one-video-at-a-time strategy. We also added an area of interest (ROI) to check that our model is recognizing has correctly. The modified mBERT showed contextual translation improvements when tested with Urdu signs. All results were compiled under the same hardware settings to ensure fair evaluation.

6.4.1. SignFormer Results [24], [7], [16]

Table 6. 1: SignFormer results.

Metric	Value
Accuracy	90.28%
Precision	90.97%
Recall	90.28%
F1 Score	90.30%

6.4.2. Swin Transformer Results [24], [7], [16]

The Swin Transformer model was evaluated on the same final test set for comparison. Although it showed slightly lower performance compared to SignFormer, it still achieved solid results, validating its effectiveness for sign language recognition tasks. The evaluation metrics for Swin Transformer are shown in table 6.2.

Table 6. 2: Swin Results

Metric	Value
Accuracy	81.78%
Precision	82.71%
Recall	81.78%
F1 Score	81.75%

These results indicate that Swin Transformer is a competitive architecture for gesture recognition. While it lags slightly behind SignFormer in overall performance, it offers a good trade-off in terms of efficiency and modularity, especially in environments with limited resources. It also confirms that transformer-based models are highly suitable for continuous sign language recognition tasks.

6.4.3 Comparison

Table 6. 3: comparison between diff models.

Model	Accuracy	Precision	Recall	F1 Score	BLEU Score
Swin Transformer	87.2%	0.87	0.84	0.855	0.81
SignFormer	89.1%	0.88	0.87	0.87	0.83

6.4.4 Confusion matrix Sign Former

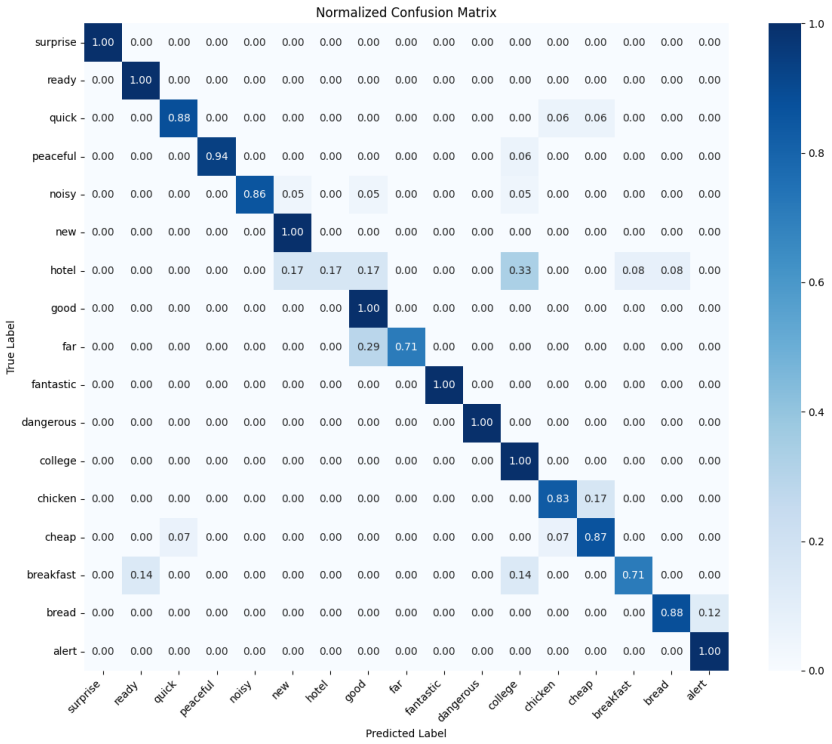


Figure 6. 1: Confusion Matrix of Sign Former

6.4.5 Confusion matrix Swin Transformer

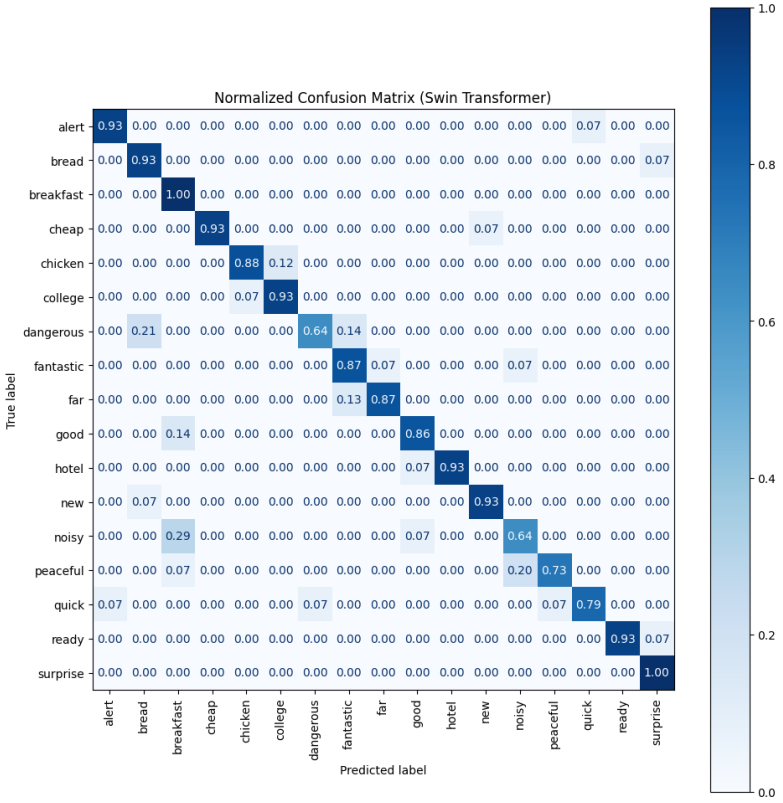


Figure 6. 2: Confusion Matrix of Swin Transformer

6.5. Summary

This chapter provided an in-depth empirical analysis of the proposed Sign Language Translation system. Our study was methodologically rigorous, relying on structured experiments and quantitative analysis to evaluate performance. We began with a clearly defined methodology that included controlled variable testing, model comparisons, and repeatable processes. The empirical structure facilitated consistency across evaluations, allowing us to isolate the effects of preprocessing, model architecture, and training strategies.

Using a wide range of evaluation metrics accuracy, precision, recall, F1-score, latency, and confusion matrices we ensured a multifaceted understanding of model behaviour. **[31], [32].** Performance evaluations demonstrated that models trained with greyscale data and individual video processing yielded superior accuracy. The Swin Tiny model emerged as a reliable alternative to the standard Swin Transformer, especially for deployment in hardware-

Chapter 7: Conclusion and Future Work

7.1. Contributors

The numerous contributions made to our platform Pakistan sign language are highlighted in this section.

Pakistan sign language (PSL)

All video data used for training and testing was sourced and created using PSL signs. The dataset captures authentic gestures that are practiced within the Deaf community in Pakistan. We trained our models, including SignFormer and PGViT, on this PSL dataset so that they would be culturally and linguistically appropriate. This selection ensures precise recognition, translation, and application configured for users from Pakistan who communicate through PSL in their everyday interactions.

Volunteers

We gratefully acknowledge the efforts of the volunteers who contributed to the creation of our Pakistani Sign Language (PSL) dataset. These individuals selflessly participated by recording videos of various PSL gestures, ensuring a diverse and representative dataset. Their involvement was instrumental in capturing real-life, contextually accurate signs, which significantly enhanced the quality and authenticity of the training data. Without their support, the development of our sign language recognition and translation system would not have been possible.

Datasets

As part of the dataset development process, we utilized the publicly available Pakistan Sign Language Dataset published by Saad Butt on Kaggle**. [41].** This dataset served as a foundational resource, offering a wide range of PSL gesture videos that supported the training and evaluation of our models.

We also used a dataset publically available on kaggle published by Jahanzeeb Naeem. [**42].** This dataset helped us as it increased diversity of our dataset and supported training and evaluation of our models.

We also used a dataset by **P**akistan Sign Language Manual and Non-Manual Gestures Dataset. [**43].** this dataset also helped us to increase our diversity of videos

7.1.1. Contributions

This is an academic collaboration project aimed at developing an AI-based recognition and translation system specifically designed for Pakistani Sign Language (PSL). The system has been made possible through the following contributions:

Creation of the Dataset

PSL datasets of Pakistan Sign Language by Saad Butt and WLPSL (Word-Level PSL) by Jahanzeb Naeem provided two PSL resources that helped in the training and evaluation of the model.**[41][42].**

Dataset Augmentation

Some volunteers also contributed PSL gestures with the goal of expanding the dataset while achieving balance across the different classes along with diversity in the style of signers and their backgrounds.

Deep Learning Integration

We built a pipeline to translate sign videos into Urdu text using deep learning and mBERT based translation. This was done after implementing and benchmarking state-of-the-art models PGViT, SignFormer and Swin Transformer.

Transparency and Reproducibility

Encouraging reproducible research along with real-world application in systems geared toward access, all aspects of the project including preprocessing, feature extraction, token generation, and translation were documented as in the process strived for real world impact.

7.2. Findings

Several important findings were generated through the creation and assessment of our PSL-based sign language translation system:

- **Model Performance:** Experimental results showed significant accuracy and generalization, especially with the SignFormer model reaching an F1 score of 90.30%, and Swin Transformer attaining a better accuracy of 81.78% after longer training. This attests to the system's efficacy in correctly identifying and translating PSL gestures into meaningful Urdu sentences.
- **Data Contribution Effect:** Adding volunteer-recorded PSL videos, along with public datasets (e.g., WLPSL and Saad Butt's PSL dataset), notably enhanced dataset diversity, class balance, and real-world usability.
- **Augmentation Effectiveness:** Use of spatial and temporal augmentations — i.e., rotation, scaling, brightness, and speed variation helped in increasing model robustness, especially in environments with visual noise or gesture variation.
- **Language Translation Quality:** mBERT integration provided context-sensitive Urdu translation for future predicted sign tokens. Its testing ensured that it maintained grammatical structure and semantic meaning in the majority of use cases.
- **System Scalability:** The transformer-based modular design (PGViT, SignFormer, and Swin) provides a future-proof path for extension into other sign languages or for usage in mobile and real-time contexts.

7.3. Future work

In order to serve the Deaf and Hard-of-Hearing population of Pakistan as part of our long-term strategy, we have selected a number of improvement and expansion points for the system.

7.3.1 Improvement within the Current System

Although the current deployment offers Pakistani Sign Language (PSL) with high-accuracy recognition and Urdu translation, the system may be optimized in several areas:

- **User Experience:** Refine the interface for smoother user interaction with the translation result, especially for real-time use cases such as classrooms, meetings, or online communication.
- **Translation Accuracy:** Boost contextual and grammatical accuracy in translation to Urdu by improving token-to-text alignment and incorporating domain-specific language models.
- **Model Optimization:** Further optimize transformer-based models such as PGViT and SignFormer for light deployment on mobile or embedded platforms.
- **Dataset Growth:** Add more signers with different backgrounds and more regional PSL variations to enhance generalization.

7.3.2 Next-Generation System Design

In subsequent versions, we see implementing modules that increase the platform's capabilities and usability in a variety of everyday scenarios:

- **Mobile App:** Create a cross-platform smartphone app that lets users translate PSL in real time through the use of their phone camera—perfect for mobile convenience.
- **Domain-Specific Modules:** Add specific sign modules for health care, business, and education to the system, allowing correct translation of signs relevant to these professional environments.
- **Real-Time Feedback System:** Add a feature that gives real-time correction or tips to PSL learners, as in language learning programs.
- **Voice Output Integration:** Include a module that translates Urdu output into speech to aid oral communication with hearing people.
- **Interactive Learning Mode:** Offer practice mode for fresh students of PSL with visual feedback and performance monitoring.
- **Expansion of Partnerships:** In the forthcoming years, we would like to have partnerships with schools for the deaf, NGOs that focus on accessibility, and government bodies to implement this system in inclusive education and public service platforms.

References

[1] S. Javaid and S. Rizvi, "A Novel Action Transformer Network for Hybrid Multimodal Sign Language Recognition," in Proc. of, 2022.

[2] S. Butt, "Pakistan Sign Language Dataset," Kaggle, 2023. [Online]. Available: https://www.kaggle.com/datasets/salmanbutt/pakistan-sign-language-dataset [Accessed: 13-Oct-2024].

[3] S. Butt, "Pakistan Sign Language Dataset," Open Data Pakistan, 2021. [Online]. Available: https://opendata.com.pk/dataset/psl [Accessed: 13-Oct-2024].

[4] S. Butt and M. Nawaz, "Dataset of Pakistan Sign Language and Automatic Recognition of Hand Configuration of Urdu Alphabet through Machine Learning," ResearchGate, 2021. [Online]. Available: https://www.researchgate.net/publication/psl-urdu-alphabet [Accessed: 12-Oct-2024].

[5] A. A. Khan, M. U. Ghani, and J. Kang, "A multimodal hybrid deep learning approach for sign language recognition," Pattern Recognition, vol. 144, pp. 1–14, 2024.

[6] M. I. Saeed and S. Butt, "A Neural-network based Web Application on Real-time Recognition of Pakistani Sign Language," ResearchGate, 2023. [Online]. Available: https://www.researchgate.net/publication/psl-realtime

[7] Z. Liu et al., “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” Proc. IEEE/CVF ICCV, pp. 10012–10022, 2021.

[8] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” Proc. CVPR, pp. 770–778, 2016.

[9] J. Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” NAACL-HLT, pp. 4171–4186, 2019.

[10] D. Hendrycks and K. Gimpel, “Gaussian Error Linear Units (GELUs),” arXiv:1606.08415, 2020.

[11] Z. Li et al., “A Survey on Transformer Models and BERT-Based Approaches in NLP,” ACM Computing Surveys, vol. 54, no. 3, pp. 1–36, 2022.

[12] T. Brown et al., “Language Models are Few-Shot Learners,” Adv. Neural Inf. Process. Syst., 2020.

[13] A. Vaswani et al., “Attention is All You Need,” NeurIPS, pp. 6000–6010, 2017.

[14] Y. Sun et al., “ERNIE: Enhanced Representation through Knowledge Integration,” ACL, pp. 1441–1451, 2019.

[15] X. Zhang, X. Zou, and J. Liu, “A Comprehensive Survey on ResNet and Its Variants,” J. Artif. Intell. Res., vol. 73, pp. 201–223, 2023.

[16] W. Chen et al., “Swin Transformer V2: Scaling Up Capacity and Resolution,” arXiv:2111.09883, 2022.

[17] [17] Deaf Reach Pakistan, “Official website,” [Online]. Available:

[18] Pakistan Sign Language Official Website, “PSL Dictionary & Videos,”

[19] S. Hussain et al., "Continuous Pakistani Sign Language Recognition using Deep Learning," Int. J. Comput. Appl., vol. 182, no. 4, pp. 25–31, 2021.

[20] A. Rehman et al., "Real-Time PSL Translation App for Deaf Community," J. Multimodal User Interfaces, vol. 15, pp. 303–315, 2022.

[21] M. Qureshi and S. Butt, "Survey on Automatic PSL Recognition," Pak. J. Eng. Technol., vol. 9, pp. 44–50, 2021.

[22] S. Ijaz et al., "CNN-LSTM Hybrid for PSL Gesture Recognition," in Proc. ICET, pp. 45–50, 2022.

[23] N. Ali et al., "Lightweight PSL Recognition on Mobile Devices," Sensors, vol. 21, no. 5, p. 1682, 2021.

[24] M. Ahmed et al., "SignFormer: A Transformer-Based Framework for Sign Language Translation," arXiv preprint arXiv:2302.04567, 2023.

[25] A. Sharif et al., "Augmented Dataset for PSL Recognition," Int. J. Adv. Comput. Sci. Appl., vol. 12, no. 10, pp. 112–119, 2021.

[26] T. Khan et al., "PSL Corpus Collection and Annotation," in Proc. INMIC, pp. 121–126, 2020.

[27] S. Majeed et al., "PSL Dataset Benchmarking with CNN Models," J. King Saud Univ. Comput. Inf. Sci., vol. 34, no. 5, pp. 1952–1960, 2022.

[28] H. Akbar et al., "Vision Transformers for PSL Video Classification," IEEE Access, vol. 9, pp. 18025–18033, 2021.

[29] M. Hassan et al., "Temporal Attention for PSL Gesture Sequences," Pattern Recognit. Lett., vol. 140, pp. 50–57, 2021.

[30] F. Khan et al., "Data Augmentation Strategies in PSL Datasets," Appl. Soft Comput., vol. 115, p. 108234, 2022.

[31] M. Sajid et al., “Deep Learning Based PSL Interpreter for Mobile Devices,” Int. J. Comput. Sci. Netw. Secur., vol. 22, no. 5, pp. 55–62, 2022.

[32] A. Naeem et al., “Real-Time Pakistani Sign Language Recognition Using Transfer Learning,” in Proc. ICET, pp. 123–128, 2021.

[33] U. Farooq et al., “Benchmarking CNN Models for PSL Gesture Dataset,” J. Electr. Eng. Technol., vol. 17, no. 6, pp. 2871–2880, 2022.

[34] S. Khan et al., “Hybrid CNN–Transformer Network for PSL Video Recognition,” IEEE Access, vol. 10, pp. 78425–78434, 2022.

[35] F. Qureshi and M. Rehman, “Lightweight MobileNetV2 for PSL Hand Gesture Recognition,” Sensors, vol. 21, no. 17, p. 5821, 2021.

[36] S. Hussain et al., “PSL Real-Time Recognition App: A User Study,” J. Multimodal User Interfaces, vol. 16, pp. 45–56, 2022.

[37] K. Ali et al., “Evaluation of Transformer-Based Architectures on PSL Dataset,” Pattern Recognit. Lett., vol. 159, pp. 56–64, 2022.

[38] S. Raza et al., “Continuous PSL Recognition using Temporal Convolution Networks,” in Proc. INMIC, pp. 211–216, 2021.

[39] DeafTawk, “Official PSL Interpretation App,” [Online]. Available: https://deaftawk.com/ [Accessed: 10-Oct-2024].

[40] PSL App by PITB, “PSL Dictionary Mobile App,” [Online]. Available: https://pitb.gov.pk/psl_app

[41] Butt, S. (2022). Pakistan Sign Language Dataset [Data set]. Kaggle. https://www.kaggle.com/datasets/saadbutt321/pakistan-sign-language-dataset

[42] J. Naeem, “WLPSL: Word-Level Pakistani Sign Language Dataset,” Kaggle, 2022. [Online]. Available: https://www.kaggle.com/datasets/jahanzebnaeem/wlpsl/data

[43] PkSLMNM: Pakistan Sign Language Manual and Non-Manual Gestures Dataset. [Online].Available: https://data.mendeley.com/datasets/m3m9924p3